

Using Python on cRIO / NI Linux RT

(!! To Enable Vibe Coding !!)



NI Team



Paco Maldonado
National Account Manager
paco.maldonado@emerson.com
(505) 377-7229
Albuquerque, NM



Leonard Nava
Account Manager
leonard.nava@emerson.com
(512) 683-0104
Austin, TX



Austen Gardner
Account Manager
austen.gardner@emerson.com
(913) 904-2192
Kansas City, MO



Luke Graham
Chief Field Applications Engineer
luke.graham@emerson.com
(505) 550-9370
Albuquerque, NM



Kelly Rael
Field Applications Engineer
kelly.rael@emerson.com
(505) 549-2855
Albuquerque, NM



Daniel Eaton
Chief Field Applications Engineer
daniel.eaton@emerson.com
(989) 272-2796
Saginaw, MI



Meet Dan



20

Years at NI

2

Years in Industry

22

Years LabVIEW

Background & Skills

- Chief FAE; Certified LabVIEW Architect; Certified LabVIEW Embedded Developer
- Software Dev, Systems Management, HiL, Protocols, CompactRIO, & FPGAs
- Previous life automating automotive test cells

+ New chat

🔍 Search

Ctrl+K

🛠️ Customize

💬 Chats

📁 Projects

📦 Artifacts

Recents

Printing variables in Python ...

Reading analog input with NI DAQ...

Create code snippets

D Dan

Pro plan

⬇️

⬆️

Printing variables in Python ▾

want to resume a previously running FPGA.

- Context manager (`with`) — always use `with nifpga.Session(...)` so the session is properly closed and the FPGA is left in a clean state even if an exception occurs.
- The NI-RIO driver must be installed on your system in addition to the Python package.

ALL PYTHON CODE WAS VIBED...

DON'T TELL MY BOSS... Great. Now make it so this application reads the U64 indicator continuously

python

```
import nifpga
import time
import signal
import sys

def run_fpga_continuous(
    bitfile_path,
    resource,
    indicator_name,
    control_name,
    control_value=True,
    poll_interval=0.1
):
    """
```

Reply...

+

Sonnet 4.6 ▾

🎤

Using Python on CompactRIO (NI Linux RT)

cRIO Basics

What is it / Where is it going?

Python on cRIO / NI Linux RT

Controller Setup

NI DAQmx

Scan Engine

FPGA

All of them together!

Today's Focus on cRIO → All Applies to PXI RT

NI Linux RT Runs on PXI too!

PXIe-8862 Details Configure a system in		
Part Numbers (13)	788817-01	787987-33
	Active	Active
	Buy or quote	Buy or quote
NI Lead Time ?	12-13 weeks Need it faster?	12-13 weeks Need it faster?
Controller OS ?	Windows 11	LabVIEW Real-Time (NI Linux Real-Time)
Processor Core ?	Intel Core i7	Intel Core i7
Maximum Controller Bandwidth ?	16 GB/s	16 GB/s

Broad Modular Instrumentation Portfolio

DAQ and Control

Multifunction I/O

Counter/Timer/Clock

Digital I/O

Analog Input/Output

Vision and Motion

FPGA/Reconfigurable I/O

Instrumentation

Oscilloscopes

High-Speed Digital I/O

DMM and SMU

Signal Generators

Switching

RF Analyzers and Generators

Interfaces

GPIO, USB, LAN

RS232/RS485

CAN, LIN, DeviceNet

SCSI, Ethernet

VXI/VME

Boundary Scan/JTAG



cRIO

What is it?

CompactRIO

A rugged, embedded computing platform with modular I/O, programmable FPGA, and a real-time OS all powered by LabVIEW.



Features

- NI Linux Real-Time OS
- Up to 1.91 GHz quad-core processors
- LabVIEW-programmable Xilinx FPGAs
- -40 °C to 70 °C operating temperature range
- 50 g shock and 5 g vibration operating range
- Over 200 modules for electrical signals, sensors, and communication protocols.
- Up to 16 GB of on-board SSD for data logging
- Computer ports including video out, USB, and Ethernet for peripherals and communication
- LabVIEW programmable main processor and FPGA

Controller and Backplane

Controller

- Provides compute infrastructure for running DAQ and control applications
- Comprises typical “motherboard” components (CPU, memory, disk etc.)

Backplane

- Provides infrastructure for connecting compute resources (controller) to IO resources (C Series modules)
- Includes DAQ ASIC and user programmable FPGA



Basic Measurements (Full Catalog)



NI 9223

1 MHz, 4 independent ch
+/- 10V



NI 9206

250 kHz, 32 muxed chs
+/- 10V



NI 9220

100 kHz, 16 independent ch
+/- 10V



NI 9208

500 kHz, 16 muxed chs
+/- 20 mA

LabVIEW has Analysis and Control Blocks

- PID
- Fuzzy Control
- Digital Filtering

Basic Generation (Full Catalog)



NI 9263

100 kHz, 4 independent ch
+/- 10V



NI 9264

25 kHz, 16 independent ch
+/- 10V



NI 9262

1 MHz, 6 independent ch
+/- 10V



NI 9265

100 kHz, 4 independent ch
0-20 mA

LabVIEW had Analysis and Control Blocks

- PID
- Fuzzy Control
- Digital Filtering

Modules for Temperature Measurement (Full Catalog)



NI 9213

High-density, most popular TC input module



NI 9214

High-density like NI 9213, but more accurate



NI 9212

Ch-Ch isolation, miniature TC connection option, best accuracy (screw terminal option)



NI 9219

Universal module with Ch-Ch Isolation

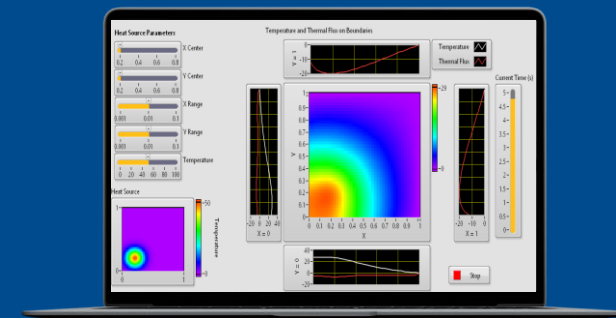


NI 9210

Miniature TC input (mini jack), lowest cost

LabVIEW Analysis Blocks for:

- Interpolation and Extrapolation
- Fitting
- Probability and Statistics



PDE THERMAL DISTRIBUTION.VI EXAMPLE

Modules for Power Measurement (Full Catalog)

Voltage



NI 9242/44

3-phase inputs (L1,L2,L3,N)
for 220/440 VAC networks



NI 9225

300 V_{rms}, ch-ch isolated

Current



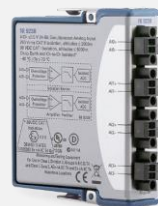
NI 9227

Good accuracy for low current
(5 Arms) measurements
Minimal over-range



NI 9245/46

Use for CTs with 5A secondary
outputs and large over-range

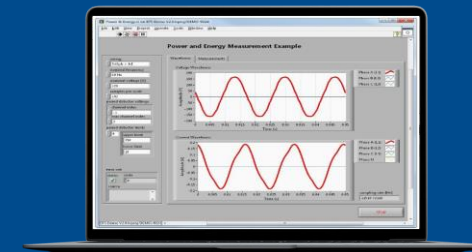


NI 9238/29

Use with CTs that have a low-
voltage output (0.333V or 10V)

Electrical Power Suite for LabVIEW

- Signal processing palette
- Waveform measurements palette
- Dynamic signal filtering



POWER AND ENERGY MEASUREMENT VI
DEVELOPED USING THE ELECTRICAL POWER SUITE

Modules for Sound and Vibration

Full Catalog



NI 9234
Most popular IEPE module



NI 9250
Designed for
NVH applications



NI 9231
High channel-count



NI 9230
Low-cost version of NI 9232

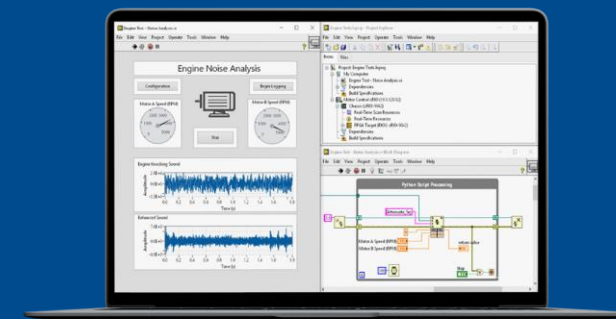


NI 9232
Designed for turbomachinery.
Higher speed with $\pm 30V$ range.

- All sound and vibration modules have:
- 24-bit resolution input circuitry
 - Built-in excitation for sensors (IEPE)
 - Sample rates between 12ks/s/ch and 102.4 ks/s/ch

Sound and Vibration Toolkit for LabVIEW

- Example Programs with S&V specific user interface
- Analysis functions for S&V



Modules for Load/Force, Pressure, Strain, Torque (Full Catalog)



NI 9237

4-Channel, 24-bit
50kS/s/ch
1/2, 1/4, Full Bridge



NI 9219

Universal module
with Ch-Ch Isolation

LabVIEW has Analysis
Blocks for:

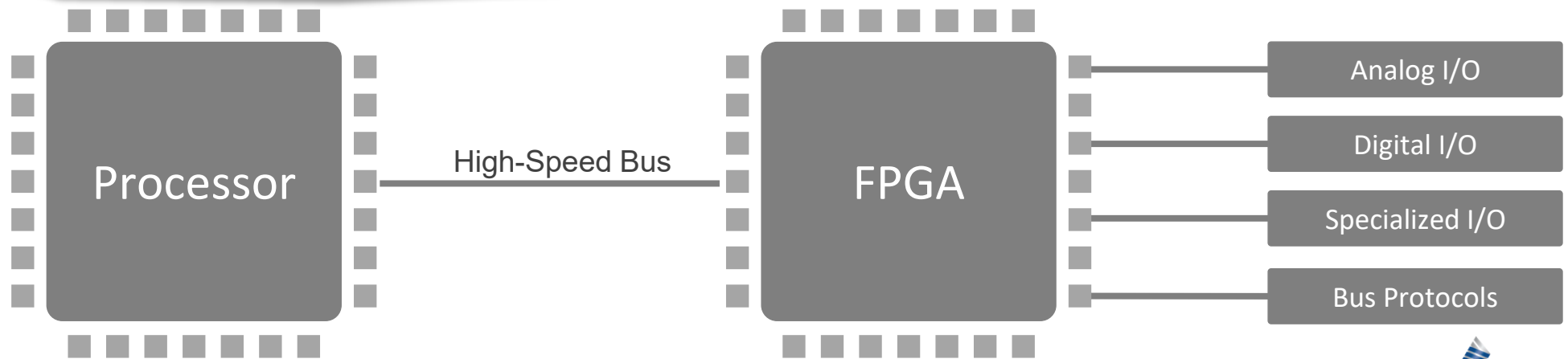
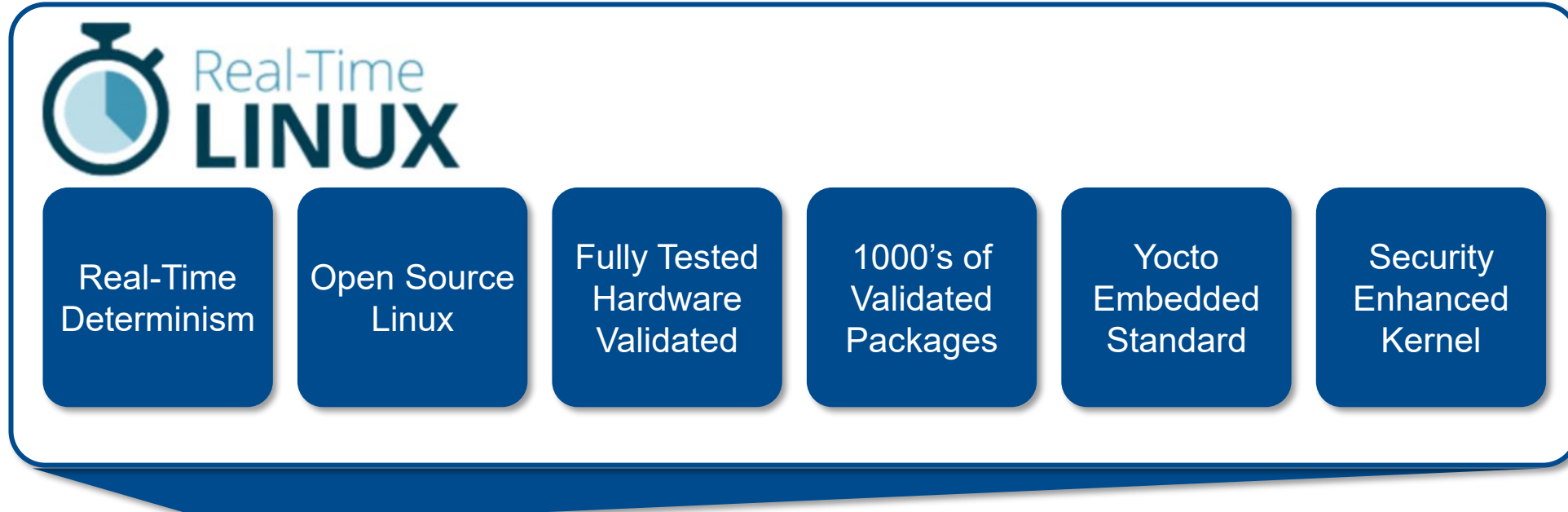
- Interpolation and Extrapolation
- Fitting
- Probability and Statistics



NI 9235/9236

8-Channel, 24-bit
50kS/s/ch
1/4 bridge 120/350 Ohm Variants

Open, Deterministic OS



Open, Deterministic OS

Open Source

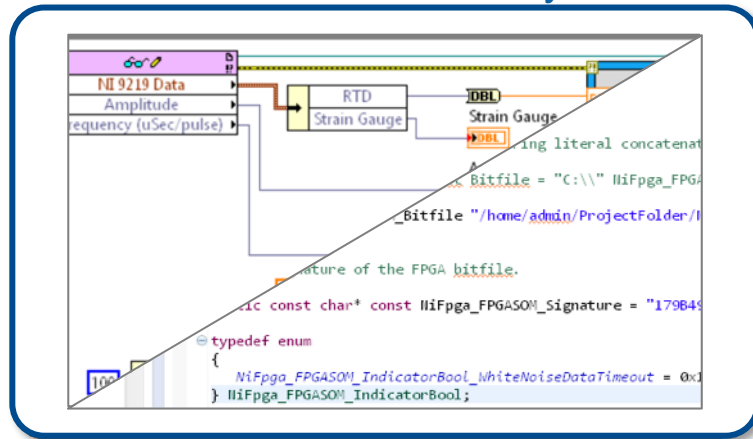
Repo:

<https://github.com/ni/linux>

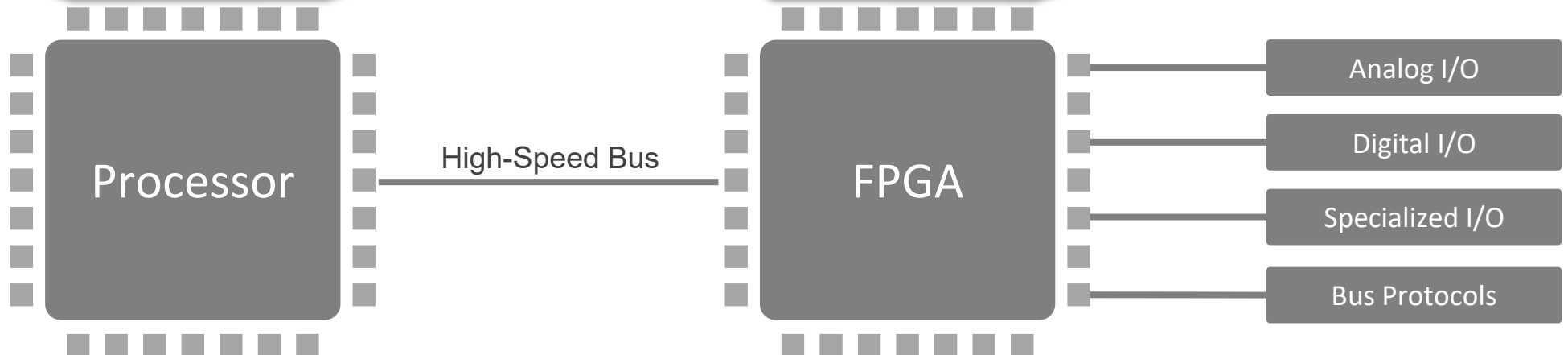
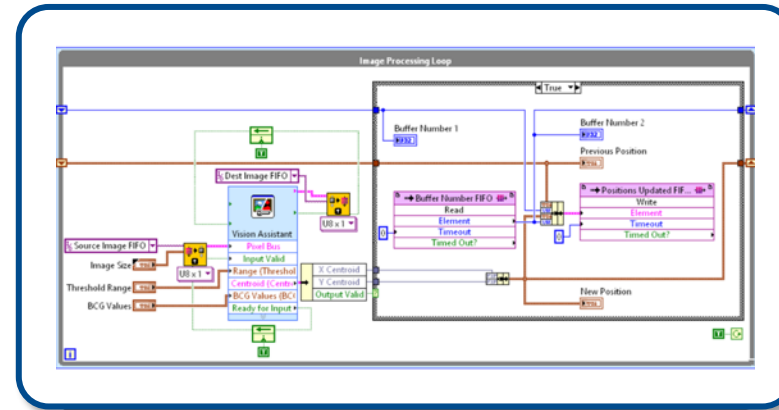


Application Software for the CompactRIO Platform

LabVIEW or C/C++, Python



LabVIEW FPGA



cRIO & sbRIO Roadmap

Unreleased product features/capabilities and technology explorations are subject to change without notice. Feedback is encouraged.

Next Generation CompactRIO & Single-Board RIO Refresh

NI has planned refreshes for both CompactRIO and Single-Board RIO

- Refresh highest performance controllers first (cRIO-904x)

Key focus areas for next generation remain consistent with our past

- Continuous performance improvements
- New features to support evolving trends

Additional focus to maintain compatibility with existing products

- Optimize for re-use within existing system architectures
- Continue to support key software programming paradigms and features



Next Generation CompactRIO Refresh

Exploration Areas for Product Enhancements

- Improved processor performance
- Improved FPGA performance
- Security enhancements
- AI
- Ease of Use

Additional Notes

- Compatibility with existing form factor (size, mounting, etc)
- Maintain DAQmx and FPGA programming options
- Maintain TSN support for synchronization



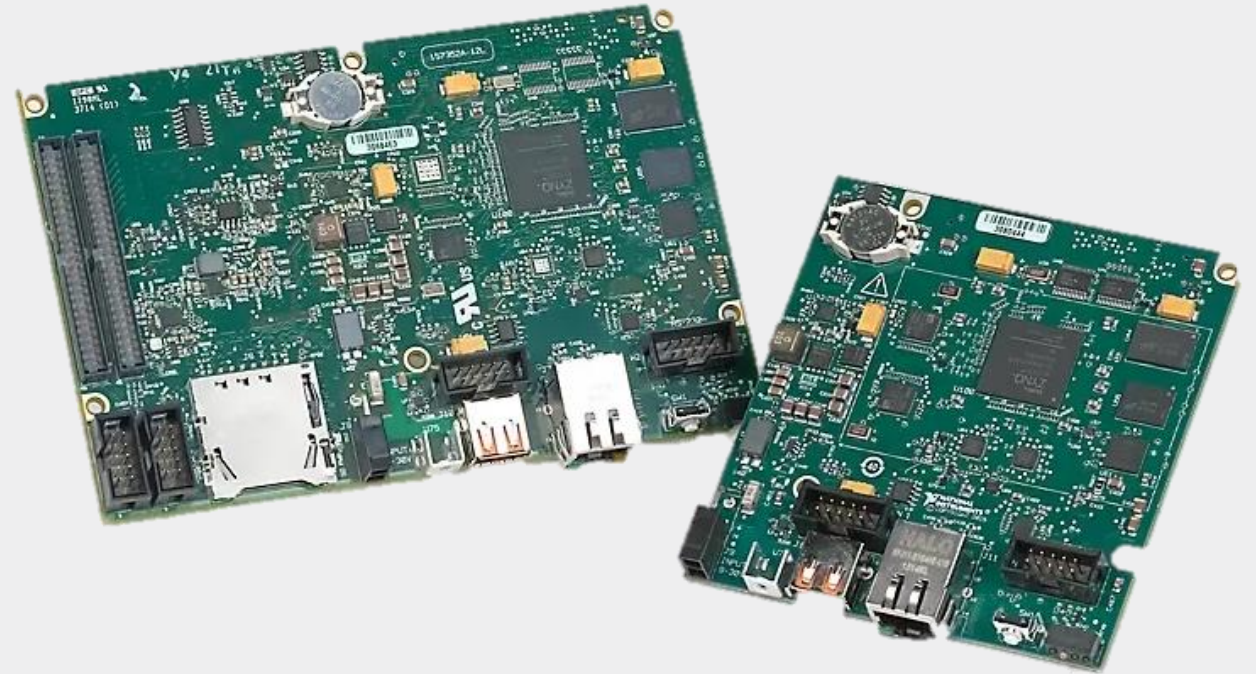
Single-Board RIO Refresh

Exploration Areas for Product Enhancements

- Improved processor performance
- Improved FPGA performance
- Security enhancements
- AI

Additional Notes

- Compatibility with existing form factor (size, mounting, etc)
- Thermal/cooling system integration requirements
- Maintain TSN support for synchronization



Python on cRIO

Assumptions

- cRIO is already:
 - Imaged
 - Has NI DAQmx drivers installed
 - Has NI RIO drivers installed
 - User name and password configured

(Start here if you need help: <https://www.ni.com/docs/en-US/bundle/crio-904x/page/getting-started-overview.html>)

- Your Laptop (Optional but helpful)
 - Has IDE & Python installed
 - Has NI DAQmx installed
 - Has LabVIEW + RT + FPGA installed (IF wanting to program FPGA directly)

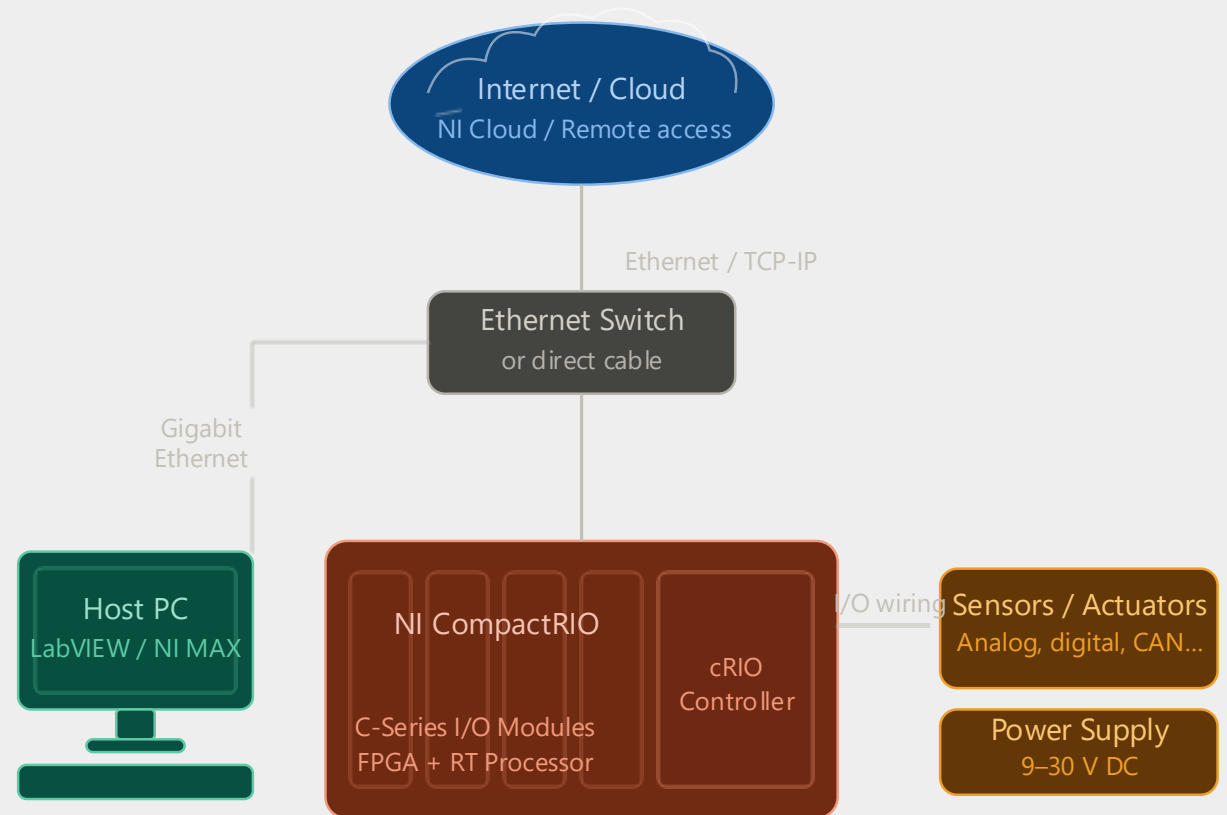
Connect cRIO to Internet & Laptop

Connect your laptop to a switch

Connect your cRIO to a switch

Make sure they are all on the same subnet (255.255.255.0)

Connect the switch to the internet (router, etc)



Install Base Image and Programming Environment

Pick your Base Image

Pick your Programming Environment

Real-Time Software - NI-cRIO-9047 (192.168.68.57)

 Base System Image Linux RT System Image 2026 Q3	 Programming Environment Other (Select to use C, C++, Python, etc.)
---	--

Real-Time Software - NI-cRIO-9047 (192.168.68.57)

Select the programming environment. [Learn more](#)

☐ LabVIEW

Version
2026

☐ LabVIEW NXG

☒ Other (Select to use C, C++, Python, etc.)

Install Required Drivers

Install NI DAQmx on Host

Install NI DAQmx on cRIO

Make sure the versions match

The screenshot displays the NI Package Manager interface. On the left, a tree view shows the system structure under 'My System' > 'Remote Systems' > 'NI-cRIO-9045-01CEEDF1'. The 'Software' section is expanded, listing various components including 'NI-DAQmx 26.0.0.49428-0+f276'. On the right, the 'INSTALLED' tab is active, showing a search for 'DAQmx' with 1 result. The result table lists the installed driver.

Name	Value
Description	Provides support for NI data acquisition and signal conditioning devices.
Version	26.0.0.49428-0+f276

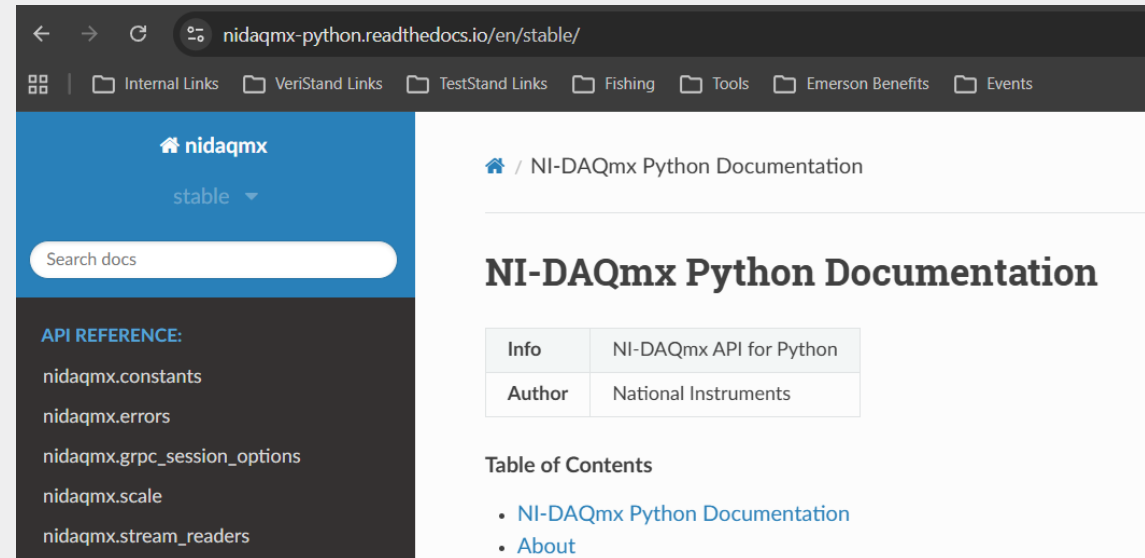
Category	Maintainer	Clear Filters	Products only	Search
1 result				
Name	Maintainer	Category	Version	
NI-DAQmx	National Instruments	Drivers	2026 Q1	

Python APIs for NI DAQmx & FPGA

Documentation = Vibe-able

DAQmx Documentation: <https://nidaqmx-python.readthedocs.io/en/stable/>

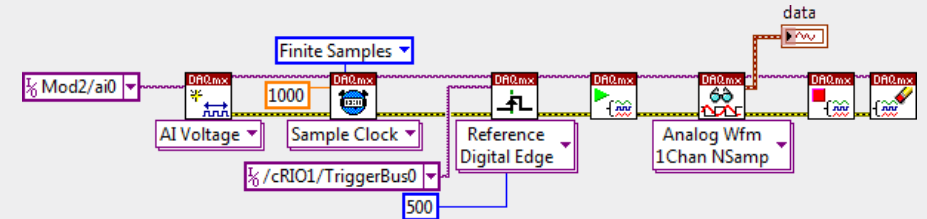
FPGA Documentation: <https://nifpga-python.readthedocs.io/en/latest/>



Pick the IO API (per slot)

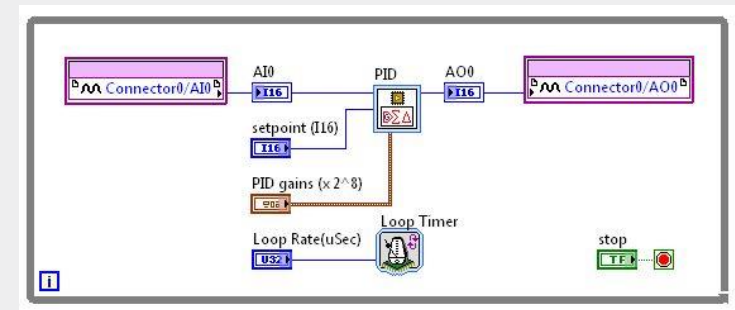
Real-Time (NI-DAQmx)

- Simple API for waveform and control from processor
- Supports control loop rates up to 5 kHz
- All channels in a task are synchronized
- Real-Time uses appropriate driver, such as NI-DAQmx or XNET



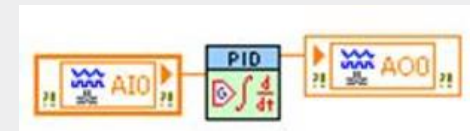
LabVIEW FPGA

- Single-point data acquisition and control from LabVIEW FPGA
- Supports control loop rates up to 10s of MHz
- Hardware-level determinism and reliability for control loops
- In-line signal processing for live filtering, calculations, and more



Real-Time Scan

- No text-based support



Set Modules to use NI-DAQmx

In Max, under cRIO, per module, set

- Program Mode = NI-DAQmx

Note:

Per module means you can mix NI DAQmx & FPGA access

The screenshot displays the NI Measurement & Automation Explorer (MAX) interface. The title bar reads "4: NI 9205 'Mod4' - Measurement & Automation Explorer". The menu bar includes File, Edit, View, Tools, and Help. The left pane shows a tree view of the system hierarchy: My System, Remote Systems, NI-cRIO-9045-01CEEDF1, Data Neighborhood, Devices and Interfaces, NI cRIO-9045 "RIO0", ASRL1::INSTR, ASRL2::INSTR, NI cRIO-9045 "cRIO1", and a list of modules. The module "4: NI 9205 'Mod4'" is selected and highlighted. The right pane shows the "Settings" tab for this module. The settings are as follows:

Setting	Value
Name	Mod4
Vendor	National Instruments
Model	NI 9205
Serial Number	0192A01C
Hardware Revision	J
Slot Number	4
Status	Present
Program Mode	Real-Time (NI-DAQmx)
Current Device Temperature	30.9°C

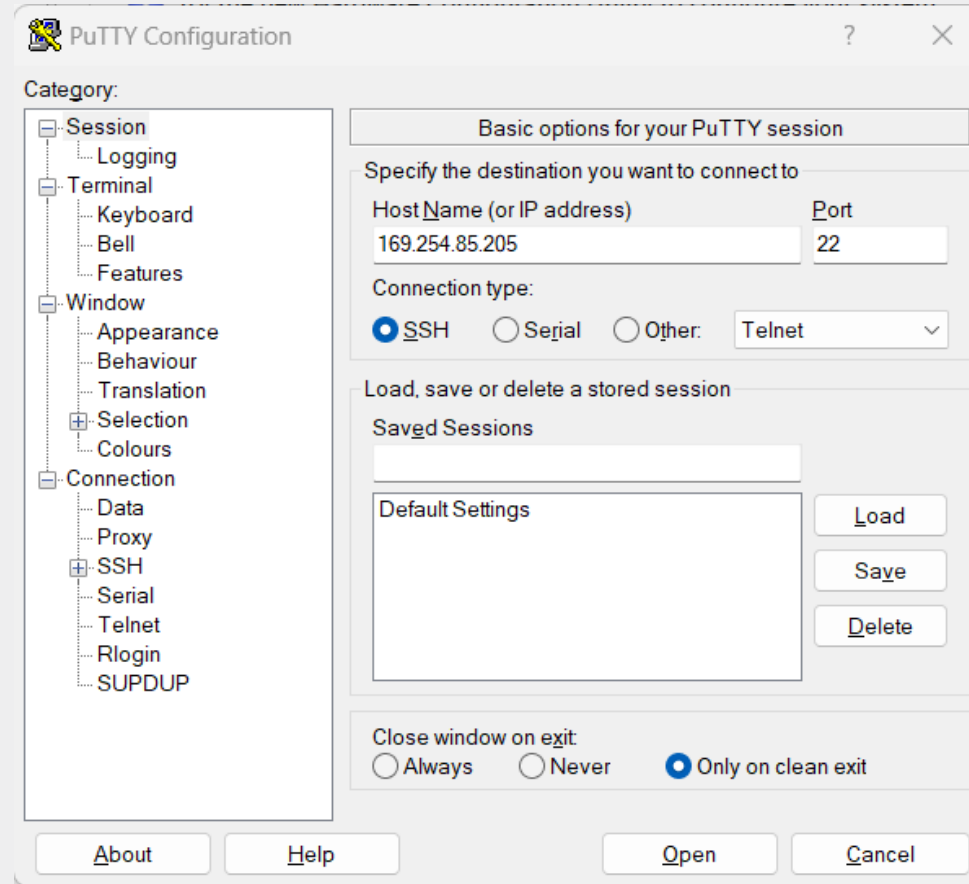
Using PuTTY (SSH) with cRIO

Connect

Set

- Host Name to cRIO IP Address
- Port to 22
- Connection type to SSH

Hit Open

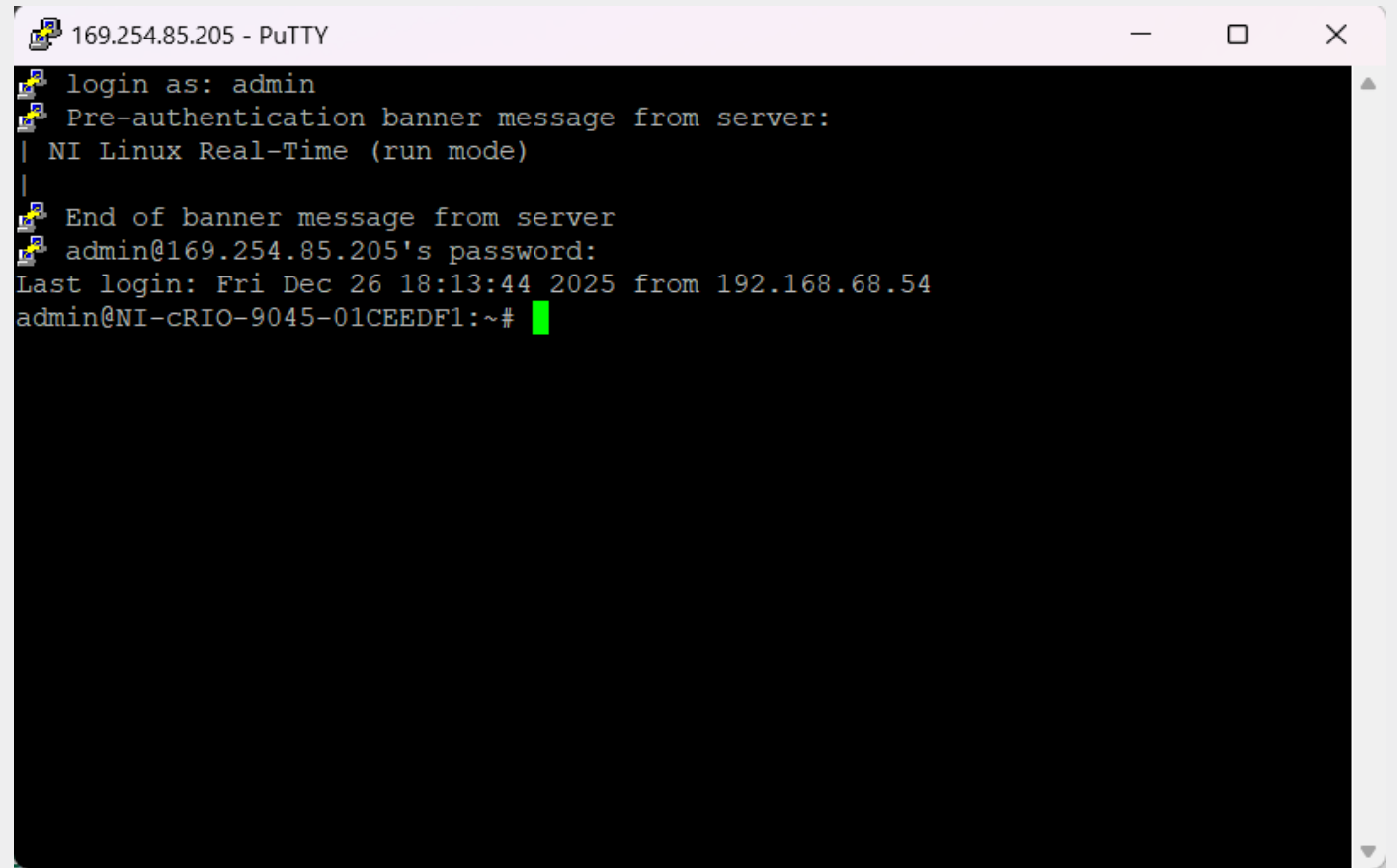


Using PuTTY (SSH) with cRIO

Log In

Enter User Name (& hit enter)

Enter Password (& hit enter)



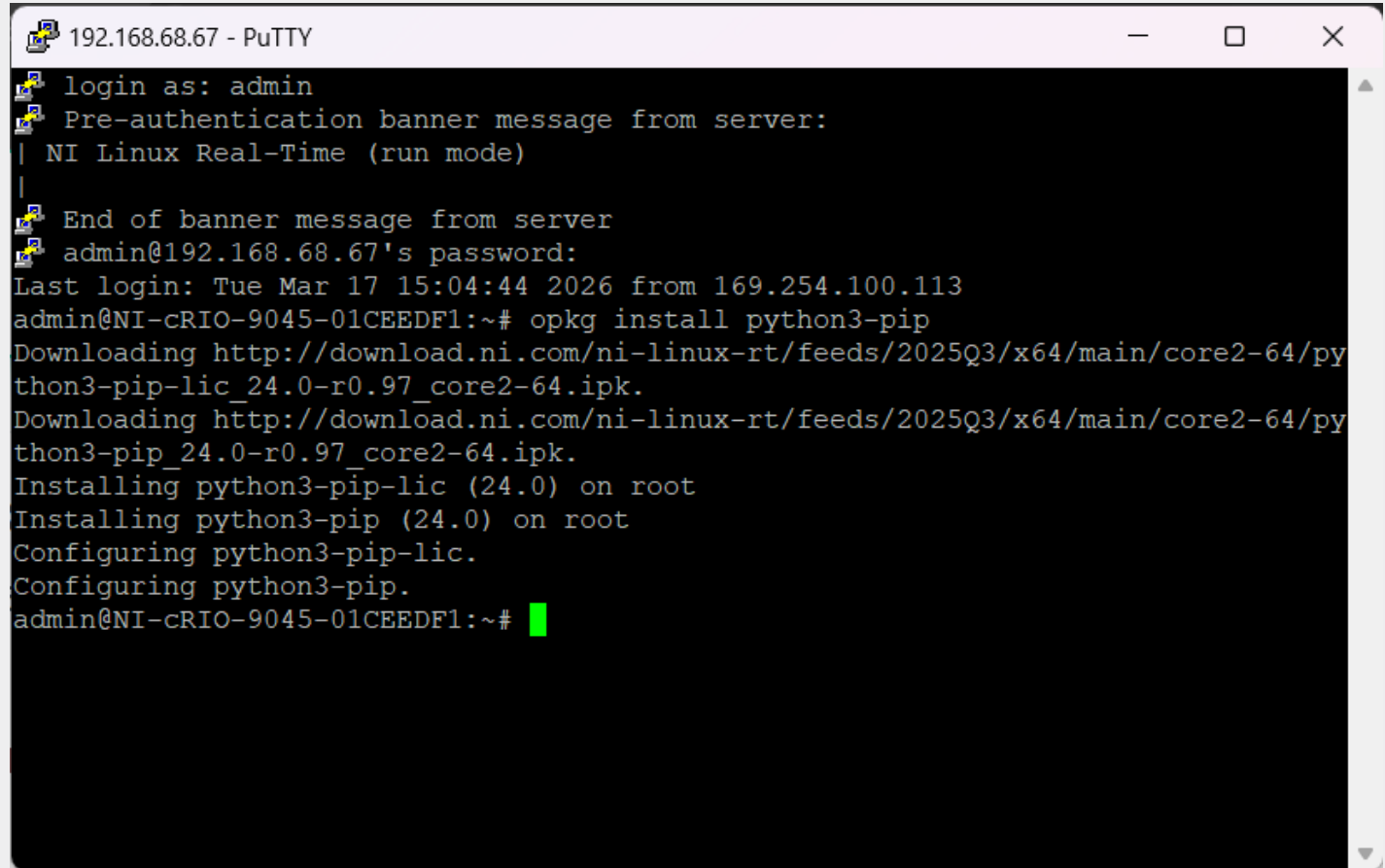
```
169.254.85.205 - PuTTY
login as: admin
Pre-authentication banner message from server:
| NI Linux Real-Time (run mode)
|
End of banner message from server
admin@169.254.85.205's password:
Last login: Fri Dec 26 18:13:44 2025 from 192.168.68.54
admin@NI-cRIO-9045-01CEEDF1:~#
```

Using PuTTY (SSH) with cRIO

Install Python

In root, type:

- `opkg install python3-pip` (Installs Python)
- Uses Python 3.12.13



```
192.168.68.67 - PuTTY
login as: admin
Pre-authentication banner message from server:
| NI Linux Real-Time (run mode)
|
End of banner message from server
admin@192.168.68.67's password:
Last login: Tue Mar 17 15:04:44 2026 from 169.254.100.113
admin@NI-cRIO-9045-01CEEDF1:~# opkg install python3-pip
Downloading http://download.ni.com/ni-linux-rt/feeds/2025Q3/x64/main/core2-64/py
thon3-pip-lic_24.0-r0.97_core2-64.ipk.
Downloading http://download.ni.com/ni-linux-rt/feeds/2025Q3/x64/main/core2-64/py
thon3-pip_24.0-r0.97_core2-64.ipk.
Installing python3-pip-lic (24.0) on root
Installing python3-pip (24.0) on root
Configuring python3-pip-lic.
Configuring python3-pip.
admin@NI-cRIO-9045-01CEEDF1:~#
```

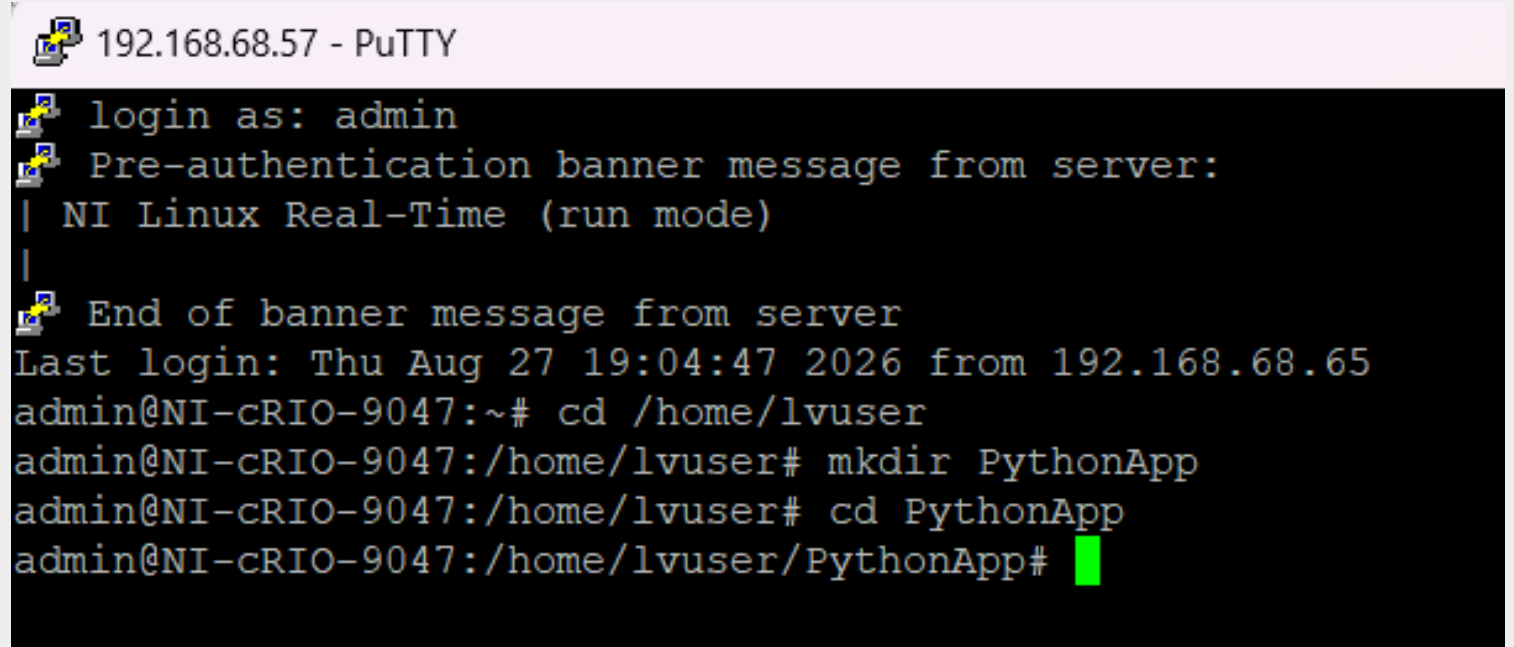
Using PuTTY (SSH) with cRIO

Create the directory

In root, type:

- `cd /home/lvuser`
- `mkdir PythonApp`
- `cd PythonApp`

Note: You should be in `/home/lvuser/PythonApp`



```
192.168.68.57 - PuTTY
login as: admin
Pre-authentication banner message from server:
| NI Linux Real-Time (run mode)
|
End of banner message from server
Last login: Thu Aug 27 19:04:47 2026 from 192.168.68.65
admin@NI-cRIO-9047:~# cd /home/lvuser
admin@NI-cRIO-9047:/home/lvuser# mkdir PythonApp
admin@NI-cRIO-9047:/home/lvuser# cd PythonApp
admin@NI-cRIO-9047:/home/lvuser/PythonApp#
```

Using PuTTY (SSH) with cRIO

Create the Virtual Environment & Activate

In /home/lvuser/PythonApp, type:

- `python -m venv venv`
- `source venv/bin/activate`

```
admin@NI-cRIO-9047:/home/lvuser/PythonApp# python -m venv venv
admin@NI-cRIO-9047:/home/lvuser/PythonApp# source venv/bin/activate
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp#
```

Note: You should now see ((venv))

Using PuTTY (SSH) with cRIO

Upgrade PIP (Package Installer)

In ((venv)) /home/lvuser/PythonApp, type:

- python -m pip install --upgrade pip

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# python -m pip install --upgrade pip
```

Using PuTTY (SSH) with cRIO

Install NI DAQmx Package

In ((venv)) /home/lvuser/PythonApp, type:

- pip install nidaqmx

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# pip install nidaqmx
Collecting nidaqmx
  Using cached nidaqmx-1.6.0-py3-none-any.whl.metadata (17 kB)
Collecting click>=8.0.0 (from nidaqmx)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting deprecation>=2.1 (from nidaqmx)
  Using cached deprecation-2.1.0-py2.py3-none-any.whl.metadata (4.6 kB)
Collecting distro>=1.9.0 (from nidaqmx)
  Using cached distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting hightime>=0.2.2 (from nidaqmx)
  Using cached hightime-1.0.0-py3-none-any.whl.metadata (2.7 kB)
Collecting nitypes>=1.0.1 (from nidaqmx)
  Using cached nitypes-1.1.0-py3-none-any.whl.metadata (7.4 kB)
Collecting numpy>=1.22 (from nidaqmx)
  Using cached numpy-2.5.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting python-decouple>=3.8 (from nidaqmx)
  Using cached python_decouple-3.8-py3-none-any.whl.metadata (14 kB)
Collecting requests>=2.25.0 (from nidaqmx)
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
```

Using PuTTY (SSH) with cRIO

Install NI FPGA Package

In ((venv)) /home/lvuser/PythonApp, type:

- pip install nifpga

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# pip install nifpga
Collecting nifpga
  Using cached nifpga-22.0.0-py3-none-any.whl.metadata (2.6 kB)
Collecting future (from nifpga)
  Using cached future-1.0.0-py3-none-any.whl.metadata (4.0 kB)
Using cached nifpga-22.0.0-py3-none-any.whl (38 kB)
Using cached future-1.0.0-py3-none-any.whl (491 kB)
Installing collected packages: future, nifpga
Successfully installed future-1.0.0 nifpga-22.0.0
```

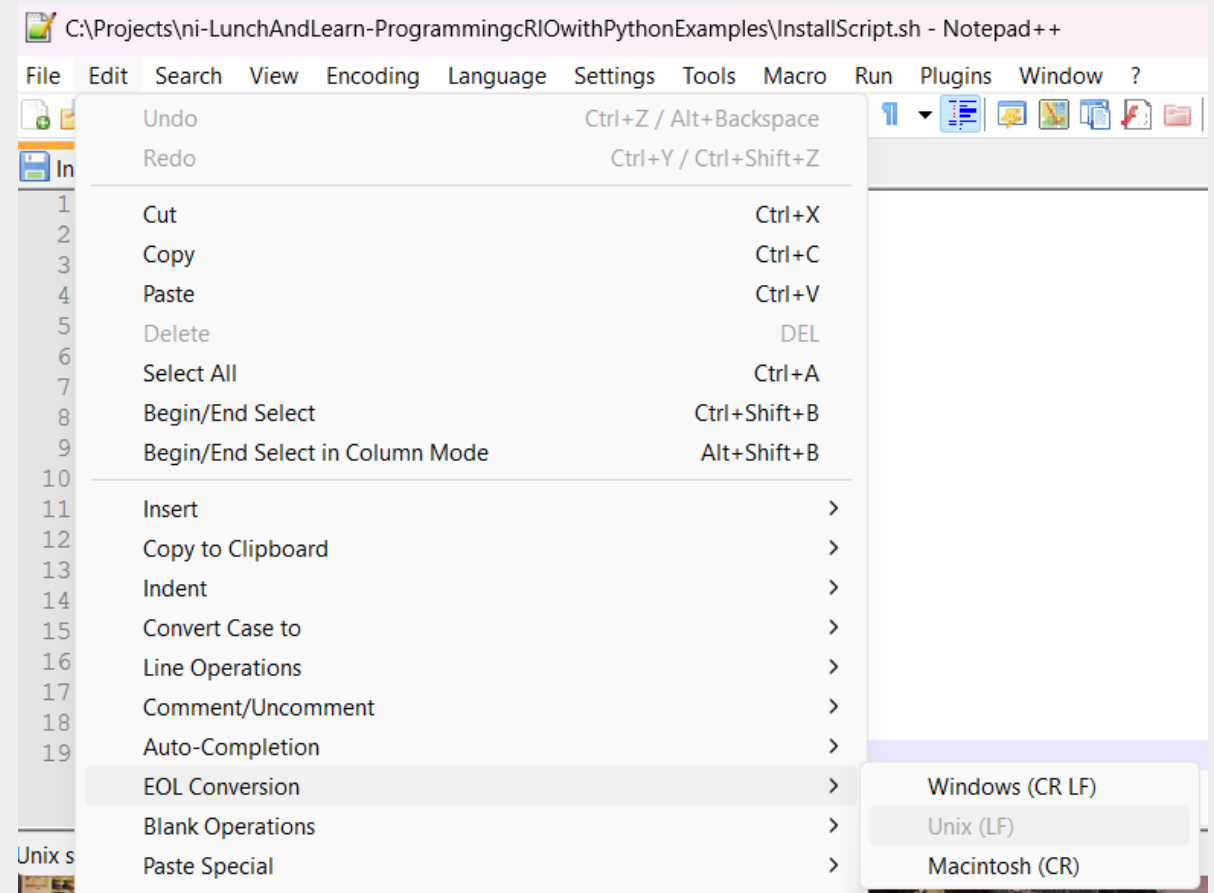
```
1  #!/bin/bash
2
3  echo "Installing Python..."
4  opkg install python3-pip
5  echo "Moving to User Directory..."
6  cd '/home/lvuser/'
7  echo "Creating New Application Directory..."
8  mkdir 'PythonApp'
9  echo "Moving to New Application Directory..."
10 cd 'PythonApp'
11 echo "Create Virtual Environment..."
12 python -m venv venv
13 echo "Activate Virtual Environment..."
14 source venv/bin/activate
15 echo "Upgrading PiP..."
16 python -m pip install --upgrade pip
17 echo "Install NI DAQmx..."
18 pip install nidaqmx
19 echo "Install NI FPGA..."
20 pip install nifpga
```

Demo: Run BASH Script

TIP: Writing a BASH File for NI Linux RT

Extension of .sh

Change EOL Conversion to Unix (LF)



DEMO: Run BASH Script

Review the BASH file (InstallScript.sh)

- Highlight setting up folder structure
- Highlight creating and activating virtual environment
- Highlight installing nidaqmx and nifpga

Use FileZilla to connect to the cRIO (Host = cRIO IP, port 22)

Move InstallScript.sh to the root/default cRIO directory

Use PuTTY to connect to the cRIO (Host = cRIO IP, port 22)

Login to cRIO via SSH (through PuTTY)

Run the script: (Type: bash InstallScript.sh)

See: Videos\Run BASH Script.mp4

```
1  #!/bin/bash
2
3  echo "Installing Python..."
4  opkg install python3-pip
5  echo "Moving to User Directory..."
6  cd '/home/lvuser/'
7  echo "Creating New Application Directory..."
8  mkdir 'PythonApp'
9  echo "Moving to New Application Directory..."
10 cd 'PythonApp'
11 echo "Create Virtual Environment..."
12 python -m venv venv
13 echo "Activate Virtual Environment..."
14 source venv/bin/activate
15 echo "Upgrading PiP..."
16 python -m pip install --upgrade pip
17 echo "Install NI DAQmx..."
18 pip install nidaqmx
19 echo "Install NI FPGA..."
20 pip install nifpga
```

DEMO: Run BASH Script Summary

```
1  #!/bin/bash
2
3  echo "Installing Python..."
4  opkg install python3-pip
5  echo "Moving to User Directory..."
6  cd '/home/lvuser/natinst/'
7  echo "Creating New Application Directory..."
8  mkdir 'PythonApp'
9  echo "Moving to New Application Directory..."
10 cd 'PythonApp'
11 echo "Create Virtual Environment..."
12 python -m venv venv
13 echo "Activate Virtual Environment..."
14 source venv/bin/activate
15 echo "Upgrading PiP..."
16 python -m pip install --upgrade pip
17 echo "Install NI DAQmx..."
18 pip install nidagmx
19 pip install nifpga
```



Run BASH files

Create a dedicated folder for Python code

Create a virtual environment & activate

Install NI DAQmx and NI FPGA Python packages

Ultimately, cRIO in (NI) Linux (RT)

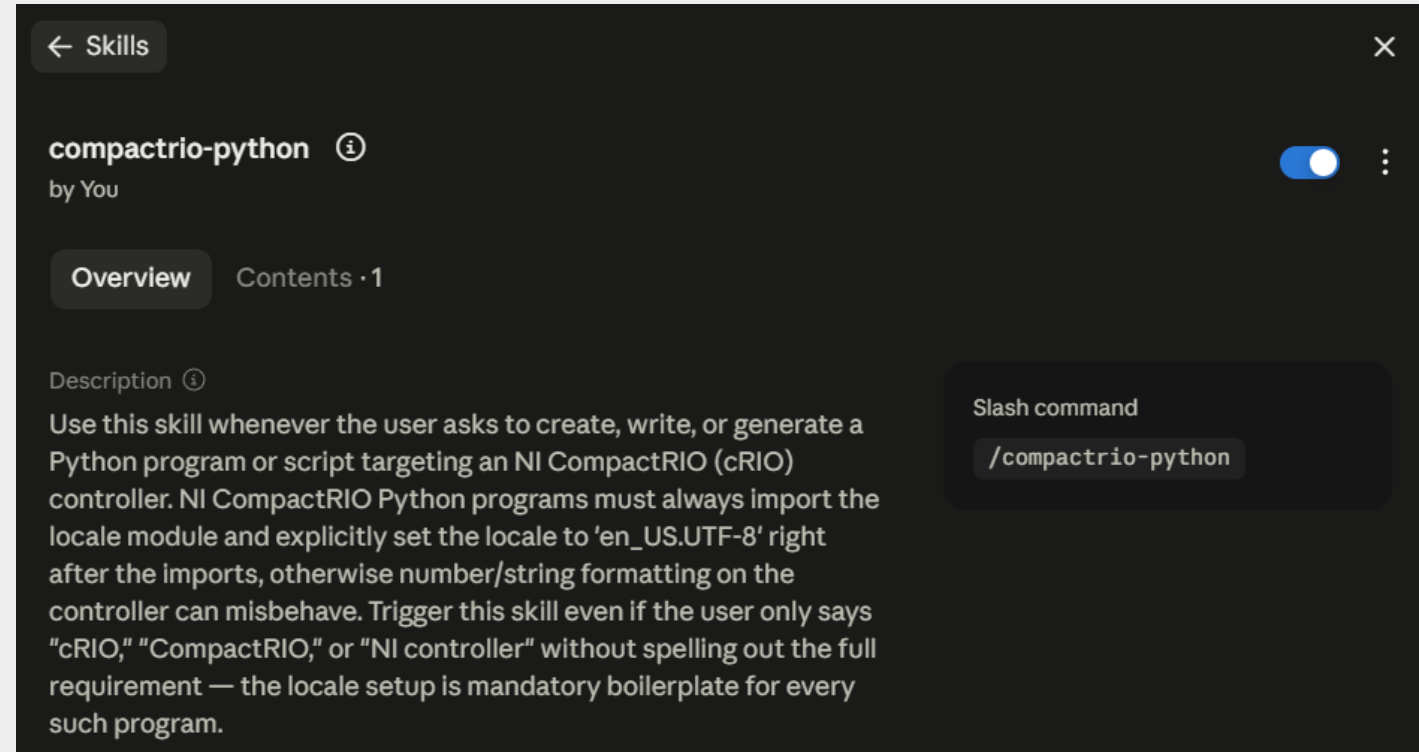
```
BasicDAQmxPythonExample.py ×
33
34 with nidaqmx.Task() as task:
35     # Add an analog input voltage channel
36     task.ai_channels.add_ai_voltage_chan(f"{mod_name}/{chan_name}")
37
38     # Configure simple timing task
39     task.timing.cfg_samp_clk_timing(rate=sample_rate, samps_per_chan=num_samples, sample_mode=AcquisitionType.FINITE)
40
41     # Allocate buffer
42     data = np.zeros(num_samples, dtype=np.float64)
43     reader = AnalogSingleChannelReader(task.in_stream)
44
45     # Start and read data
46     task.start()
47     reader.read_many_sample(data, number_of_samples_per_channel=num_samples, timeout=10.0)
48
```

Demo: Python + DAQmx on cRIO

TIP: Build skills...

For the main script, tell AI to add:

```
import locale
locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')
```



The screenshot shows a dark-themed user interface for configuring a skill. At the top, there is a back arrow and the text 'Skills', followed by a close button (X). The skill name 'compactrio-python' is displayed with an information icon (i) and a toggle switch that is currently turned on. Below the skill name, it says 'by You'. There are two tabs: 'Overview' (which is selected) and 'Contents · 1'. The 'Description' section, marked with an information icon (i), contains the following text: 'Use this skill whenever the user asks to create, write, or generate a Python program or script targeting an NI CompactRIO (cRIO) controller. NI CompactRIO Python programs must always import the locale module and explicitly set the locale to 'en_US.UTF-8' right after the imports, otherwise number/string formatting on the controller can misbehave. Trigger this skill even if the user only says "cRIO," "CompactRIO," or "NI controller" without spelling out the full requirement — the locale setup is mandatory boilerplate for every such program.' To the right of the description, there is a 'Slash command' section with a text input field containing '/compactrio-python'.

← Skills

compactrio-python ⓘ

by You

Overview Contents · 1

Description ⓘ

Use this skill whenever the user asks to create, write, or generate a Python program or script targeting an NI CompactRIO (cRIO) controller. NI CompactRIO Python programs must always import the locale module and explicitly set the locale to 'en_US.UTF-8' right after the imports, otherwise number/string formatting on the controller can misbehave. Trigger this skill even if the user only says "cRIO," "CompactRIO," or "NI controller" without spelling out the full requirement — the locale setup is mandatory boilerplate for every such program.

Slash command

/compactrio-python

Using FileZilla for File Transfer

(Manual Way)

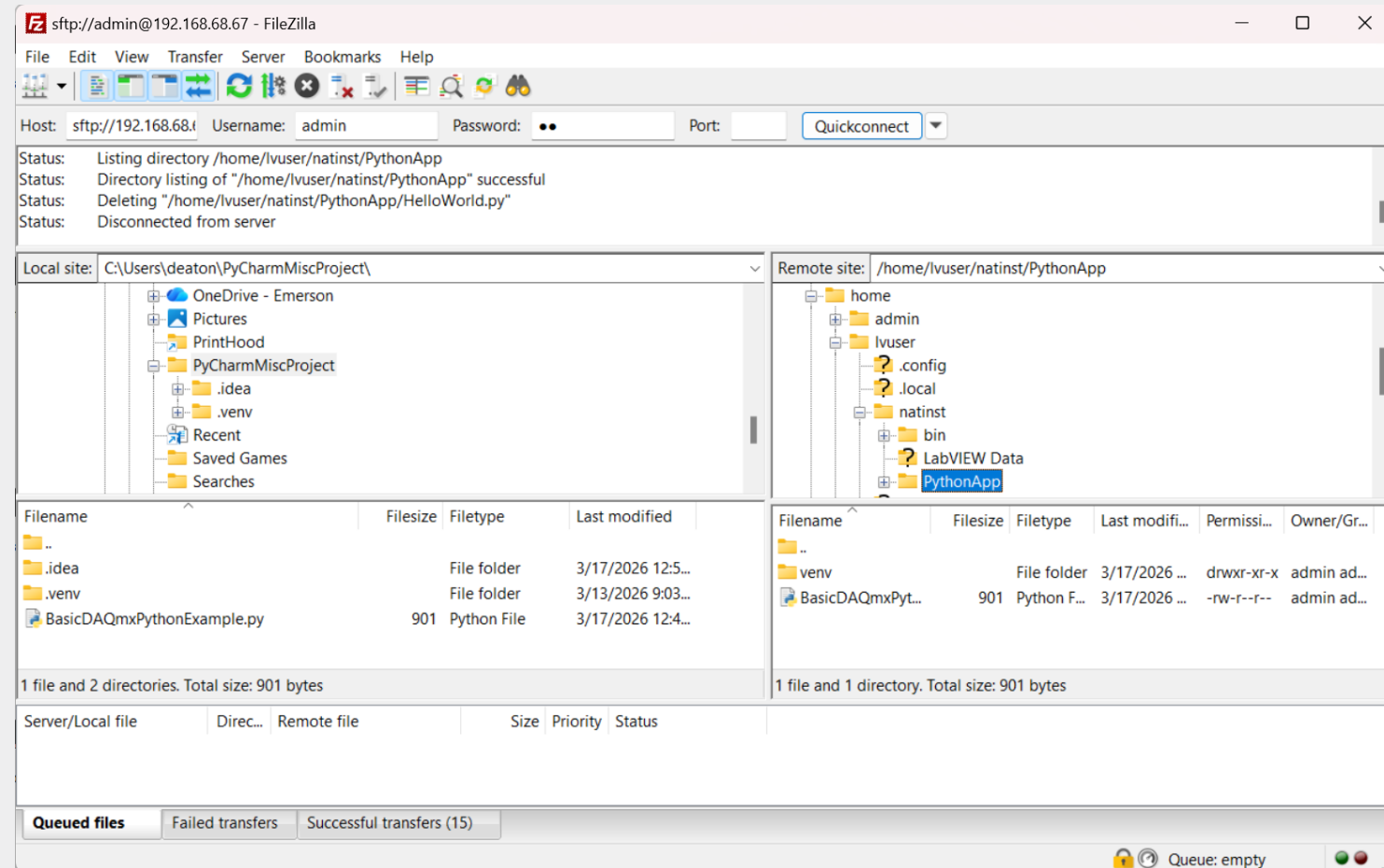
Set

- Host Name to cRIO IP Address
- Enter Username
- Enter Password
- Port to 22

Hit Quick Connect

Move python scripts to:

/home/lvuser/PythonApp



Run Python Code

(Manual Way)

In ((venv)) /home/lvuser/PythonApp, type:

- python <scriptname>.py

Using provided example:

- python BasicDAQmxPythonExample.py

See: Videos\Python + DAQmx on cRIO.mp4

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# python BasicDAQmxPythonExample.py
BasicDAQmxPythonExample.py is executing on Linux
DAQmx Version: <function version at 0x7fb271058680>
Acquired data: [-1.39289974e-04  2.48684476e-05  2.48684476e-05  2.48684476e-05
 2.48684476e-05  2.48684476e-05 -1.39289974e-04  2.48684476e-05
-1.39289974e-04 -1.39289974e-04  2.48684476e-05  2.48684476e-05
-1.39289974e-04 -1.39289974e-04  2.48684476e-05 -3.03448396e-04
 2.48684476e-05 -1.39289974e-04  2.48684476e-05 -1.39289974e-04
-1.39289974e-04  2.48684476e-05 -1.39289974e-04  2.48684476e-05
 2.48684476e-05 -1.39289974e-04  2.48684476e-05  2.48684476e-05
 2.48684476e-05 -1.39289974e-04  2.48684476e-05  2.48684476e-05
 2.48684476e-05 -1.39289974e-04 -1.39289974e-04 -1.39289974e-04
-1.39289974e-04 -1.39289974e-04 -1.39289974e-04  2.48684476e-05
-1.39289974e-04 -1.39289974e-04  2.48684476e-05 -1.39289974e-04
-1.39289974e-04 -3.03448396e-04  2.48684476e-05  2.48684476e-05
-1.39289974e-04  2.48684476e-05  2.48684476e-05  2.48684476e-05
-1.39289974e-04 -1.39289974e-04  2.48684476e-05 -1.39289974e-04
 2.48684476e-05 -1.39289974e-04 -1.39289974e-04  2.48684476e-05
2.48684476e-05 1.89026870e-04 -1.39289974e-04  2.48684476e-05
2.48684476e-05 2.48684476e-05 2.48684476e-05 2.48684476e-05
2.48684476e-05 -1.39289974e-04  2.48684476e-05 -1.39289974e-04
2.48684476e-05 2.48684476e-05 -1.39289974e-04 -3.03448396e-04
2.48684476e-05 -1.39289974e-04  2.48684476e-05  2.48684476e-05
2.48684476e-05 2.48684476e-05 2.48684476e-05 -3.03448396e-04
2.48684476e-05 -3.03448396e-04  2.48684476e-05 -1.39289974e-04]
```

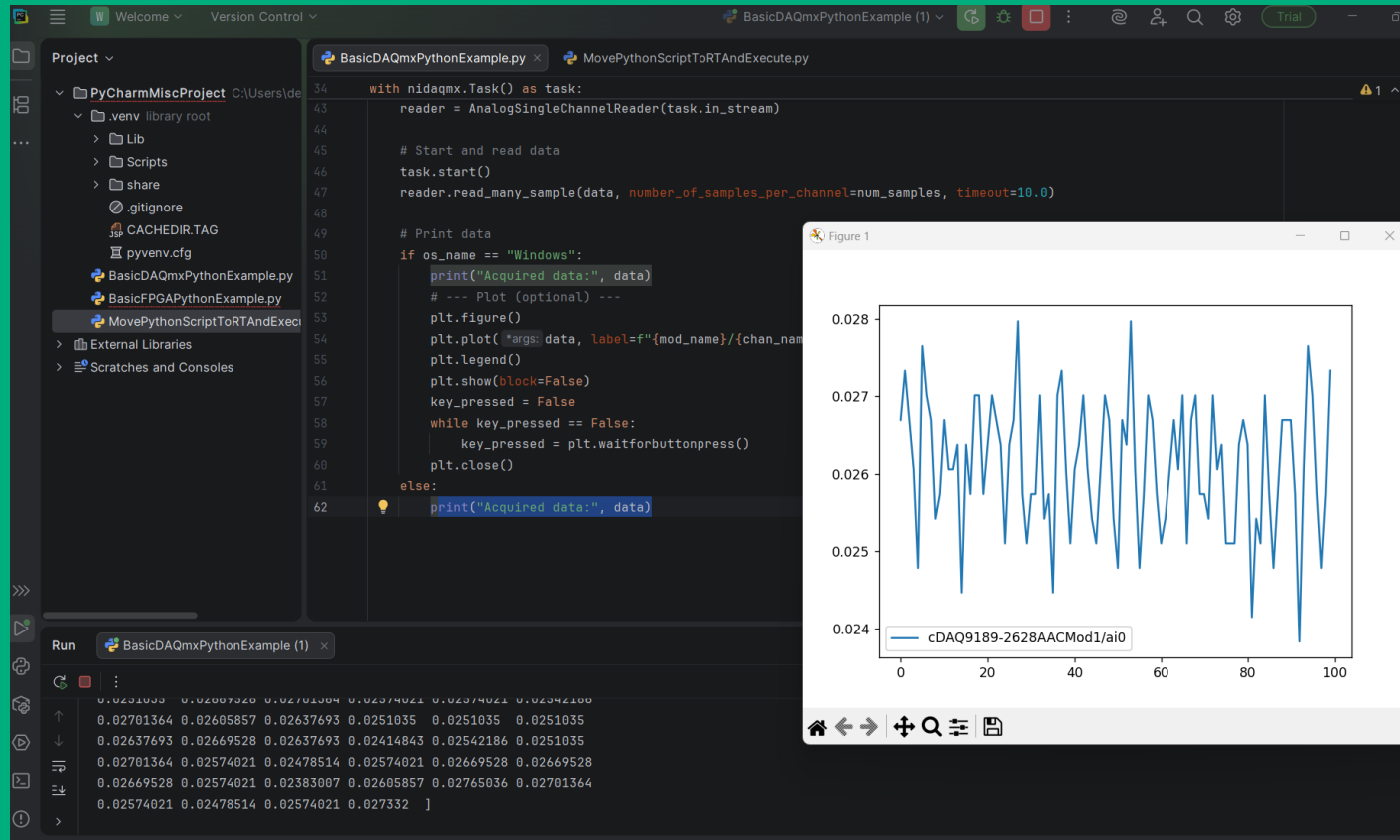
DEMO: Python + DAQmx on cRIO

Summary

It is easy to run Python on cRIO

And use NI DAQmx API

```
192.168.68.67 - PuTTY
login as: admin
Pre-authentication banner message from server:
| NI Linux Real-Time (run mode)
|
| End of banner message from server
Last login: Wed Mar 18 14:21:58 2026 from 192.168.68.64
admin@NI-cRIO-9045-01CEEDF1:~$ cd '/home/lvuser/natinst/PythonApp'
admin@NI-cRIO-9045-01CEEDF1:/home/lvuser/natinst/PythonApp$ source venv/bin/activate
((venv) ) admin@NI-cRIO-9045-01CEEDF1:/home/lvuser/natinst/PythonApp$ python BasicDAQmxPythonExample.py
BasicDAQmxPythonExample.py is executing on Linux
DAQmx Version: <function version at 0x7f2592ec0ae0>
Acquired data: [ 1.89026870e-04 -1.39289974e-04 -3.03448396e-04 -3.03448396e-04
 2.48684476e-05 2.48684476e-05 2.48684476e-05 2.48684476e-05
 2.48684476e-05 -1.39289974e-04 -1.39289974e-04 -3.03448396e-04
-1.39289974e-04 1.89026870e-04 -1.39289974e-04 -1.39289974e-04
-1.39289974e-04 2.48684476e-05 2.48684476e-05 2.48684476e-05
-1.39289974e-04 -1.39289974e-04 -1.39289974e-04 2.48684476e-05
-1.39289974e-04 -1.39289974e-04 2.48684476e-05 2.48684476e-05
-1.39289974e-04 2.48684476e-05 2.48684476e-05 -1.39289974e-04
2.48684476e-05 -1.39289974e-04 -1.39289974e-04 1.89026870e-04
2.48684476e-05 2.48684476e-05 -1.39289974e-04 -1.39289974e-04
2.48684476e-05 -1.39289974e-04 2.48684476e-05 2.48684476e-05
-1.39289974e-04 2.48684476e-05 -1.39289974e-04 -1.39289974e-04
2.48684476e-05 2.48684476e-05 -3.03448396e-04 -1.39289974e-04
2.48684476e-05 2.48684476e-05 -1.39289974e-04 2.48684476e-05
-1.39289974e-04 2.48684476e-05 1.89026870e-04 -1.39289974e-04
2.48684476e-05 2.48684476e-05 -1.39289974e-04 2.48684476e-05
-3.03448396e-04 -1.39289974e-04 2.48684476e-05 -1.39289974e-04
-1.39289974e-04 2.48684476e-05 2.48684476e-05 -1.39289974e-04
2.48684476e-05 2.48684476e-05 1.89026870e-04 -1.39289974e-04
2.48684476e-05 -1.39289974e-04 2.48684476e-05 2.48684476e-05
-1.39289974e-04 2.48684476e-05 -1.39289974e-04 -3.03448396e-04
-3.03448396e-04 -1.39289974e-04 2.48684476e-05 -3.03448396e-04
2.48684476e-05 -1.39289974e-04 2.48684476e-05 -1.39289974e-04
-1.39289974e-04 2.48684476e-05 -1.39289974e-04 1.89026870e-04
2.48684476e-05 -1.39289974e-04 -1.39289974e-04 -1.39289974e-04]
((venv) ) admin@NI-cRIO-9045-01CEEDF1:/home/lvuser/natinst/PythonApp$
```



Demo: Run Python on Windows w/ cDAQ (DAQmx Only)

TIP...Run with cDAQ on Windows First...(DAQmx Only)

Easiest Debug

```
os_name = platform.system()
if os_name == "Windows":
    import matplotlib.pyplot as plt

#Highlight NI DAQmx works on Windows and on NI Linux RT
print(f"{Path(__file__).name} is executing on {os_name}")
if os_name == "Windows":
    #Set Module Names for the debug system
    mod_name = "cDAQ9189-2628AACMod1"
elif os_name == "Linux":
    #Set Module Names for the cRIO
    mod_name = "Mod4"
```

```
# Print data
if os_name == "Windows":
    print("Acquired data:", data)
    # --- Plot (optional) ---
    plt.figure()
    plt.plot(*args: data, label=f"{mod_name}/{chan_name}")
    plt.legend()
    plt.show(block=False)
    key_pressed = False
    while key_pressed == False:
        key_pressed = plt.waitforbuttonpress()
    plt.close()
else:
    print("Acquired data:", data)
```

Debugging (DAQmx only)

cRIO is an embedded target (Limited GUI)

Windows is easier for debugging

Build cases for running the same code on Windows as on NI Linux RT

- Import based on OS
- GUI Plotting based on OS

Debug most of the code on Windows

Use a cDAQ 9189 chassis instead of cRIO chassis

- Use same NI DAQmx API
- Use C Series Modules
- Use same wiring

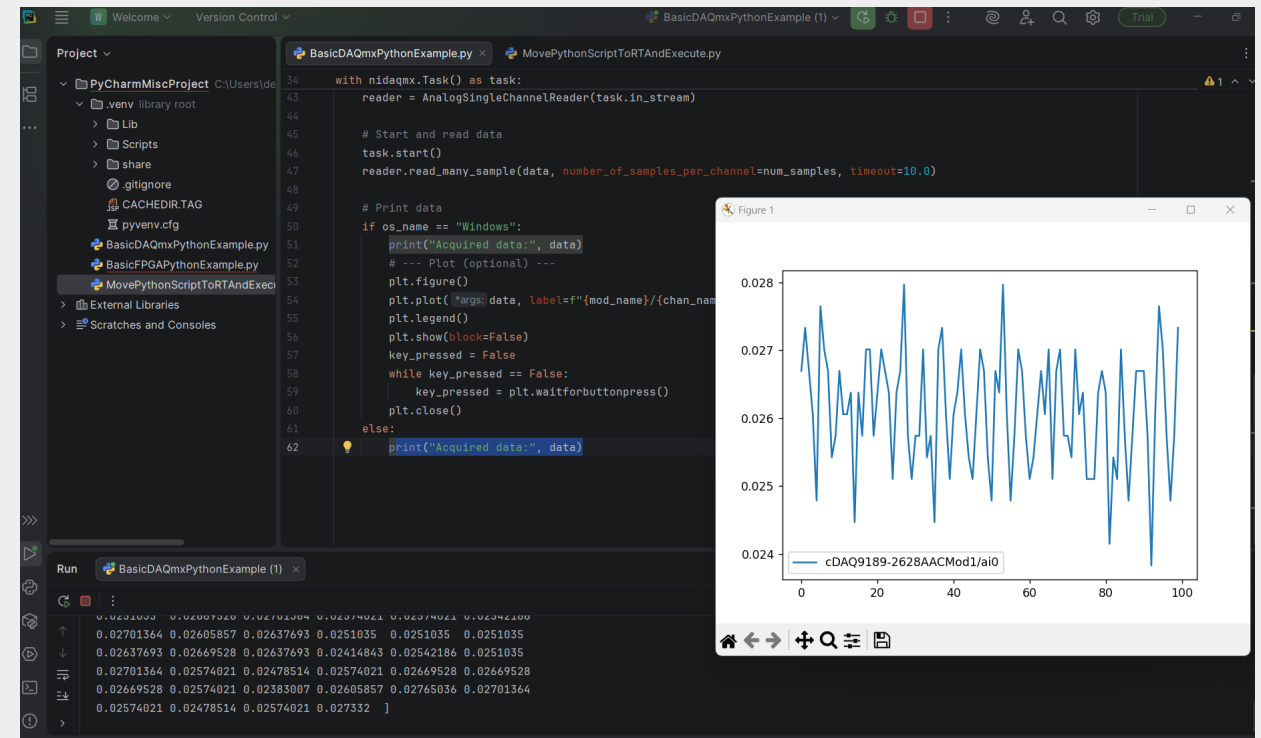
Debugging (DAQmx only)

Review NI DAQmx Python Examples\BasicDAQmxPythonExample.py

- Highlight OS based cases
 - Import
 - Plotting

Run BasicDAQmxPythonExample.py on Windows

See: Videos\Run Python on Windows with cDAQ_DAQmx Only.mp4

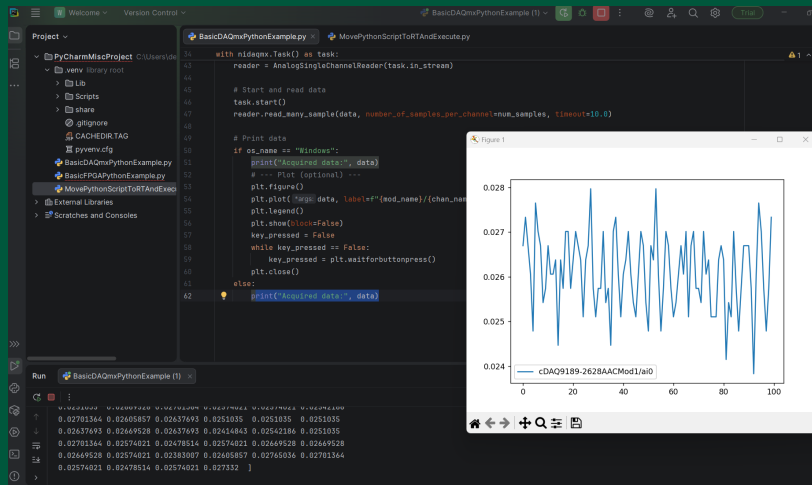


DEMO: Run Python on Windows w/ cDAQ (DAQmx only) Summary

Run the same Python script fully on Windows

Only swap cRIO Chassis for cDAQ Chassis

Configuration changes slightly, main code UNTOUCHED



```
Run MovePythonScriptToRTAndExecute x
" C:\Users\deaton\PycharmProjects\cRIO Python DAQmx Example Program\.venv\Scripts\python.exe" "C:\Users\deaton\PycharmProjects\cRIO Python DAQmx
Running on: Windows
Remote command: cd /home/lvuser/PythonApp/ && /home/lvuser/PythonApp/venv/bin/python BasicDAQmxPythonExample.py
Connecting to 192.168.68.57 using password...
Transferring 'C:\Users\deaton\PycharmProjects\cRIO Python DAQmx Example Program\BasicDAQmxPythonExample.py' -> '192.168.68.57:/home/lvuser/Pyth
Transfer verified. File size: 1885 bytes
Connected to 192.168.68.57

Changing to directory: /home/lvuser/PythonApp/
Executing script: BasicDAQmxPythonExample.py

-----
Output:
BasicDAQmxPythonExample.py is executing on Linux
DAQmx Version: <function version at 0x7f402d8785e0>
Acquired data: [ 0.4207632  0.41665923  0.4207632  0.41731587  0.4207632  0.4179725
 0.42027073  0.41912161  0.41846498 -0.42350359 -0.42432438 -0.42301111
-0.42481685 -0.42219032 -0.42596596 -0.42120537 -0.4258018  -0.421862
-0.42563764 -0.42120537 -0.4258018  -0.42169785 -0.42613012 -0.42169785
-0.42596596 -0.42268279 -0.42448854 -0.42350359 -0.42317527 -0.42416022
-0.42317527 -0.42481685 -0.42301111 -0.42563764 -0.42251864 -0.4258018
-0.42219032 -0.42596596 -0.421862  -0.42629428 -0.42120537 -0.42727922
-0.42104121 -0.4258018  -0.42169785 -0.4258018  -0.42284695 -0.42432438
-0.42498101 -0.42301111 -0.42514517 -0.42268279 -0.42498101 -0.42251864
-0.42498101 -0.42268279 -0.42613012 -0.421862  -0.42678675 -0.4207129
```

Demo: Automate Remote Workflow

Remote Development Options

Windows → NI Linux RT

Some IDEs allow automatic file sync and SSH connection

Use a python script to transfer and run your application

Demo: Automate Remote Workflow

Review NI DAQmx Python Examples\MovePythonScriptToRTAndExecute.py

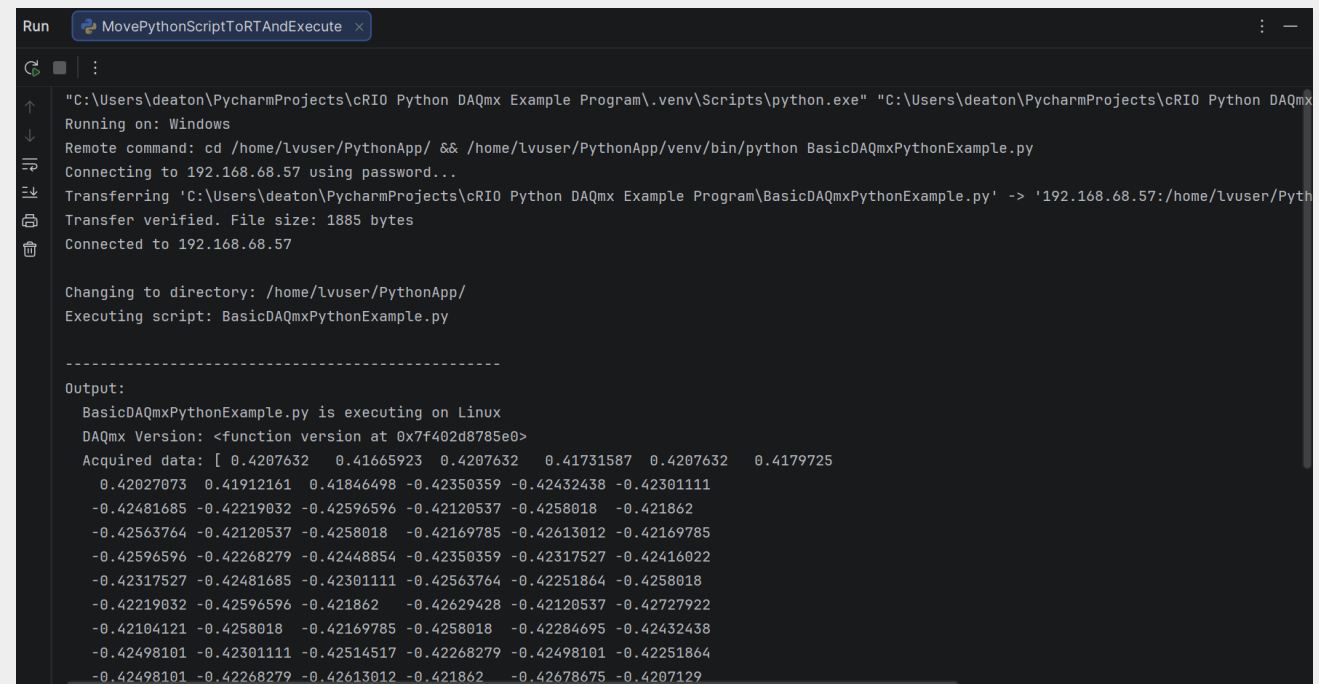
- Highlight paramiko package
- Highlight SSH
- Highlight sFTP
- Highlight remote command
- Highlight streaming back response

In FileZilla

- Connect to cRIO
- GoTo: /home/lvuser/PythonApp/
- Delete file BasicDAQmxPythonExample.py

Run MovePythonScriptToRTAndExecute.py

- Show Result
- Show File back on cRIO



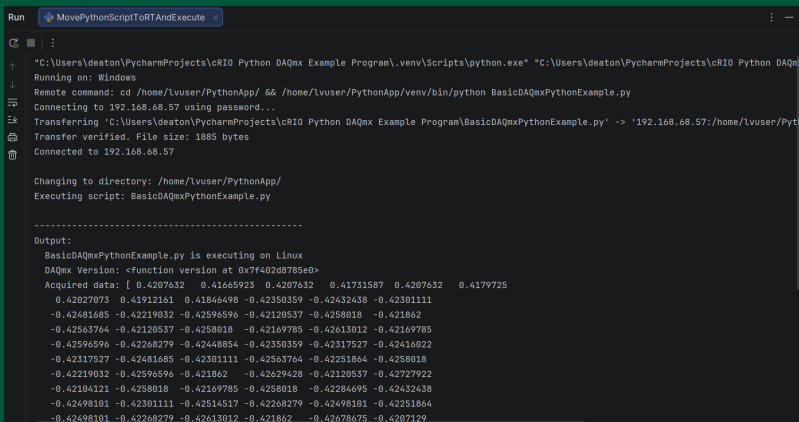
```
Run MovePythonScriptToRTAndExecute x
"C:\Users\deaton\PycharmProjects\cRIO Python DAQmx Example Program\.venv\Scripts\python.exe" "C:\Users\deaton\PycharmProjects\cRIO Python DAQmx Example Program\MovePythonScriptToRTAndExecute.py"
Running on: Windows
Remote command: cd /home/lvuser/PythonApp/ && /home/lvuser/PythonApp/venv/bin/python BasicDAQmxPythonExample.py
Connecting to 192.168.68.57 using password...
Transferring 'C:\Users\deaton\PycharmProjects\cRIO Python DAQmx Example Program\BasicDAQmxPythonExample.py' -> '192.168.68.57:/home/lvuser/PythonApp/'
Transfer verified. File size: 1885 bytes
Connected to 192.168.68.57

Changing to directory: /home/lvuser/PythonApp/
Executing script: BasicDAQmxPythonExample.py

-----
Output:
BasicDAQmxPythonExample.py is executing on Linux
DAQmx Version: <function version at 0x7f402d8785e0>
Acquired data: [ 0.4207632  0.41665923  0.4207632  0.41731587  0.4207632  0.4179725
 0.42027073  0.41912161  0.41846498 -0.42350359 -0.42432438 -0.42301111
-0.42481685 -0.42219032 -0.42596596 -0.42120537 -0.4258018  -0.421862
-0.42563764 -0.42120537 -0.4258018  -0.42169785 -0.42613012 -0.42169785
-0.42596596 -0.42268279 -0.42448854 -0.42350359 -0.42317527 -0.42416022
-0.42317527 -0.42481685 -0.42301111 -0.42563764 -0.42251864 -0.4258018
-0.42219032 -0.42596596 -0.421862  -0.42629428 -0.42120537 -0.42727922
-0.42104121 -0.4258018  -0.42169785 -0.4258018  -0.42284695 -0.42432438
-0.42498101 -0.42301111 -0.42514517 -0.42268279 -0.42498101 -0.42251864
-0.42498101 -0.42268279 -0.42613012 -0.421862  -0.42678675 -0.4207129]
```

See: Videos\Automate Remote Workflow.mp4

DEMO: Automate Remote Workflow Summary



```
Run MovePythonScriptToRTAndExecute
"C:\Users\deaton\PycharmProjects\cRIO_Python_DAQmx_Example_Program\.venv\Scripts\python.exe" "C:\Users\deaton\PycharmProjects\cRIO_Python_DAQmx_Example_Program\BasicDAQmxPythonExample.py"
Running on: Windows
Remote command: cd /home/lvuser/PythonApp/ && /home/lvuser/PythonApp/venv/bin/python BasicDAQmxPythonExample.py
Connecting to 192.168.68.57 using password...
Transferring 'C:\Users\deaton\PycharmProjects\cRIO_Python_DAQmx_Example_Program\BasicDAQmxPythonExample.py' -> '192.168.68.57:/home/lvuser/PythonApp/'
Transfer verified, File size: 1865 bytes
Connected to 192.168.68.57

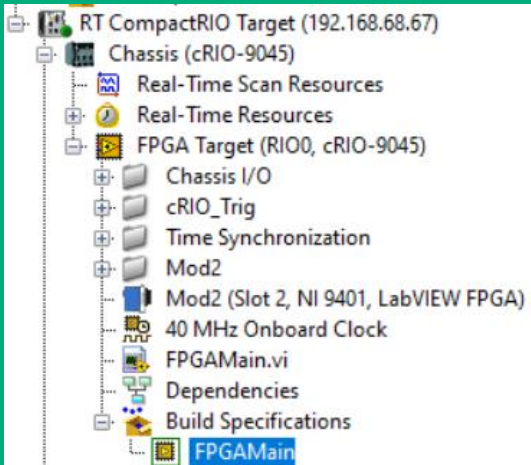
Changing to directory: /home/lvuser/PythonApp/
Executing script: BasicDAQmxPythonExample.py

-----
Output:
BasicDAQmxPythonExample.py is executing on Linux
DAQmx Version: <function version at 0x7f402d8785e0>
Acquired data: [ 0.4207632  0.41665923  0.4207632  0.41731587  0.4207632  0.4179725
 0.42077073  0.41917161  0.41846498 -0.42350359 -0.42432438 -0.42381111
-0.42481885 -0.42219832 -0.42596596 -0.42128537 -0.42588818 -0.421862
-0.42543764 -0.42128537 -0.42588818 -0.42149785 -0.42613012 -0.42169785
-0.42596596 -0.42268279 -0.42448854 -0.42350359 -0.42317527 -0.42416822
-0.42317527 -0.42481885 -0.42381111 -0.42543764 -0.42251864 -0.42588818
-0.42219832 -0.42596596 -0.421862 -0.42629428 -0.42128537 -0.42272922
-0.42184121 -0.42588818 -0.42169785 -0.42588818 -0.42284695 -0.42432438
-0.42498101 -0.42381111 -0.42514517 -0.42268279 -0.42498101 -0.42251864
-0.42498101 -0.42268279 -0.42613012 -0.421862 -0.42678675 -0.4207129
```



Automatically:

- Move Files to RT
- Run Script on RT
- All Through Windows IDE



```
((venv) ) admin@NI-CRIO-9047:/home/lvuser/PythonApp# python BasicFPGAPythonExample.py
Connecting to FPGA: RIO0
Bitfile: /home/lvuser/PythonApp/FPGAMainBitfile_9047.lvbitx
FPGA is running.
```

Timestamp	Loop#	Count	Simulated Mod2 DIO0
19:15:44	1	Count = 1	Simulated Mod2 DIO0 = True
19:15:44	2	Count = 1	Simulated Mod2 DIO0 = False
19:15:44	3	Count = 2	Simulated Mod2 DIO0 = True
19:15:44	4	Count = 2	Simulated Mod2 DIO0 = False
19:15:44	5	Count = 3	Simulated Mod2 DIO0 = True
19:15:45	6	Count = 3	Simulated Mod2 DIO0 = False
19:15:45	7	Count = 4	Simulated Mod2 DIO0 = True
19:15:45	8	Count = 4	Simulated Mod2 DIO0 = False
19:15:45	9	Count = 5	Simulated Mod2 DIO0 = True
19:15:45	10	Count = 5	Simulated Mod2 DIO0 = False
19:15:45	11	Count = 6	Simulated Mod2 DIO0 = True
19:15:45	12	Count = 6	Simulated Mod2 DIO0 = False
19:15:45	13	Count = 7	Simulated Mod2 DIO0 = True
19:15:45	14	Count = 7	Simulated Mod2 DIO0 = False
19:15:45	15	Count = 8	Simulated Mod2 DIO0 = True
19:15:46	16	Count = 8	Simulated Mod2 DIO0 = False

Demo: Python + FPGA on cRIO

Demo: Python + FPGA on cRIO

Prerequisites

Create a LabVIEW Project with your cRIO

Add the FPGA Target

Move Only Desired Modules under the FPGA (PER MODULE)

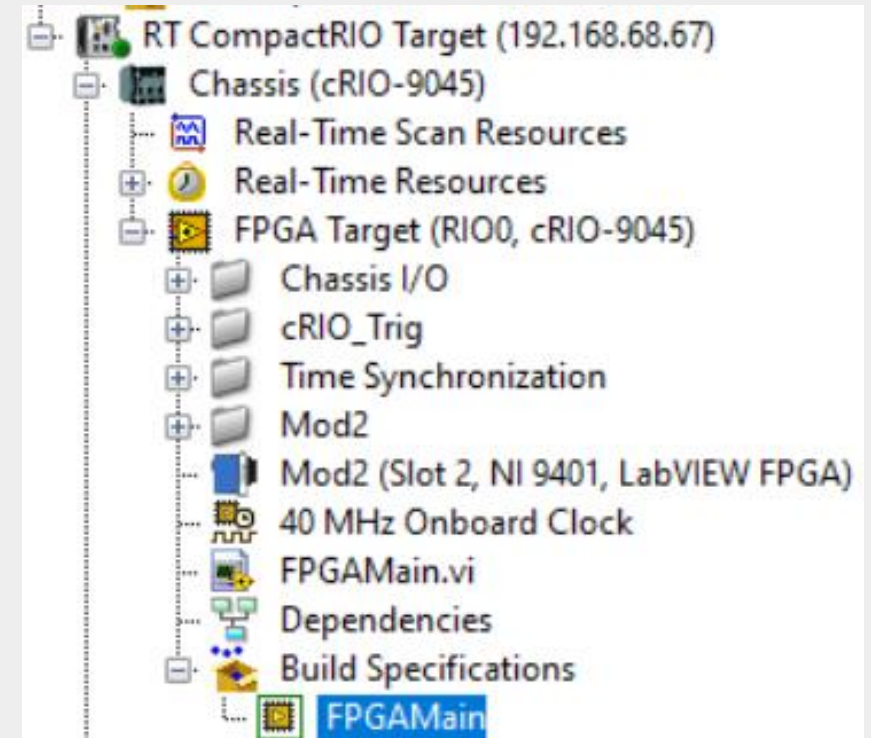
Create your LabVIEW FPGA Program

(Test your Code in Simulated Mode)

Compile your program into a bitfile

(Test your Code w/ Hardware)

Then, write your Python...



Demo: Automate Remote Workflow

Review NI FPGA Python Examples\BasicFPGAPythonExample.py

- Highlight import nifpga
- Highlight creating nifpga session
- Highlight starting/reseting fpga
- Highlight reading indicators /writing controls

Use FileZilla to connect to the cRIO (Host = cRIO IP, port 22)

- Move BasicFPGAPythonExample.py to /home/lvuser/PythonApp

Use PuTTY to connect to the cRIO (Host = cRIO IP, port 22)

- Change Directory: `cd '/home/lvuser/PythonApp'`
- Activate Virtual Env: `source venv/bin/activate`
- Run Python Script: `python BasicFPGAPythonExample.py`

See: Videos\Python + FPGA on cRIO.mp4

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# python BasicFPGAPythonExample.py
Connecting to FPGA: RIO0
Bitfile: /home/lvuser/PythonApp/FPGAMainBitfile_9047.lvbitx
FPGA is running.
```

Timestamp	Loop#	Count	Simulated Mod2 DIO0
19:15:44	1	Count = 1	Simulated Mod2 DIO0 = True
19:15:44	2	Count = 1	Simulated Mod2 DIO0 = False
19:15:44	3	Count = 2	Simulated Mod2 DIO0 = True
19:15:44	4	Count = 2	Simulated Mod2 DIO0 = False
19:15:44	5	Count = 3	Simulated Mod2 DIO0 = True
19:15:45	6	Count = 3	Simulated Mod2 DIO0 = False
19:15:45	7	Count = 4	Simulated Mod2 DIO0 = True
19:15:45	8	Count = 4	Simulated Mod2 DIO0 = False
19:15:45	9	Count = 5	Simulated Mod2 DIO0 = True
19:15:45	10	Count = 5	Simulated Mod2 DIO0 = False
19:15:45	11	Count = 6	Simulated Mod2 DIO0 = True
19:15:45	12	Count = 6	Simulated Mod2 DIO0 = False
19:15:45	13	Count = 7	Simulated Mod2 DIO0 = True
19:15:45	14	Count = 7	Simulated Mod2 DIO0 = False
19:15:45	15	Count = 8	Simulated Mod2 DIO0 = True
19:15:46	16	Count = 8	Simulated Mod2 DIO0 = False

DEMO: Automate Remote Workflow Summary



Use the cRIO FPGA in Python (PER MODULE)

```
((venv) ) admin@NI-cRIO-9047:/home/lvuser/PythonApp# python BasicFPGAPythonExample.py
Connecting to FPGA: RIO0
Bitfile: /home/lvuser/PythonApp/FPGAMainBitfile_9047.lvbitx
FPGA is running.
```

Timestamp	Loop#	Count	Simulated Mod2 DIO0
19:15:44	1	Count = 1	Simulated Mod2 DIO0 = True
19:15:44	2	Count = 1	Simulated Mod2 DIO0 = False
19:15:44	3	Count = 2	Simulated Mod2 DIO0 = True
19:15:44	4	Count = 2	Simulated Mod2 DIO0 = False
19:15:44	5	Count = 3	Simulated Mod2 DIO0 = True
19:15:45	6	Count = 3	Simulated Mod2 DIO0 = False
19:15:45	7	Count = 4	Simulated Mod2 DIO0 = True
19:15:45	8	Count = 4	Simulated Mod2 DIO0 = False
19:15:45	9	Count = 5	Simulated Mod2 DIO0 = True
19:15:45	10	Count = 5	Simulated Mod2 DIO0 = False
19:15:45	11	Count = 6	Simulated Mod2 DIO0 = True
19:15:45	12	Count = 6	Simulated Mod2 DIO0 = False
19:15:45	13	Count = 7	Simulated Mod2 DIO0 = True
19:15:45	14	Count = 7	Simulated Mod2 DIO0 = False
19:15:45	15	Count = 8	Simulated Mod2 DIO0 = True
19:15:46	16	Count = 8	Simulated Mod2 DIO0 = False

Demo: Full Application

Demo: Full Application

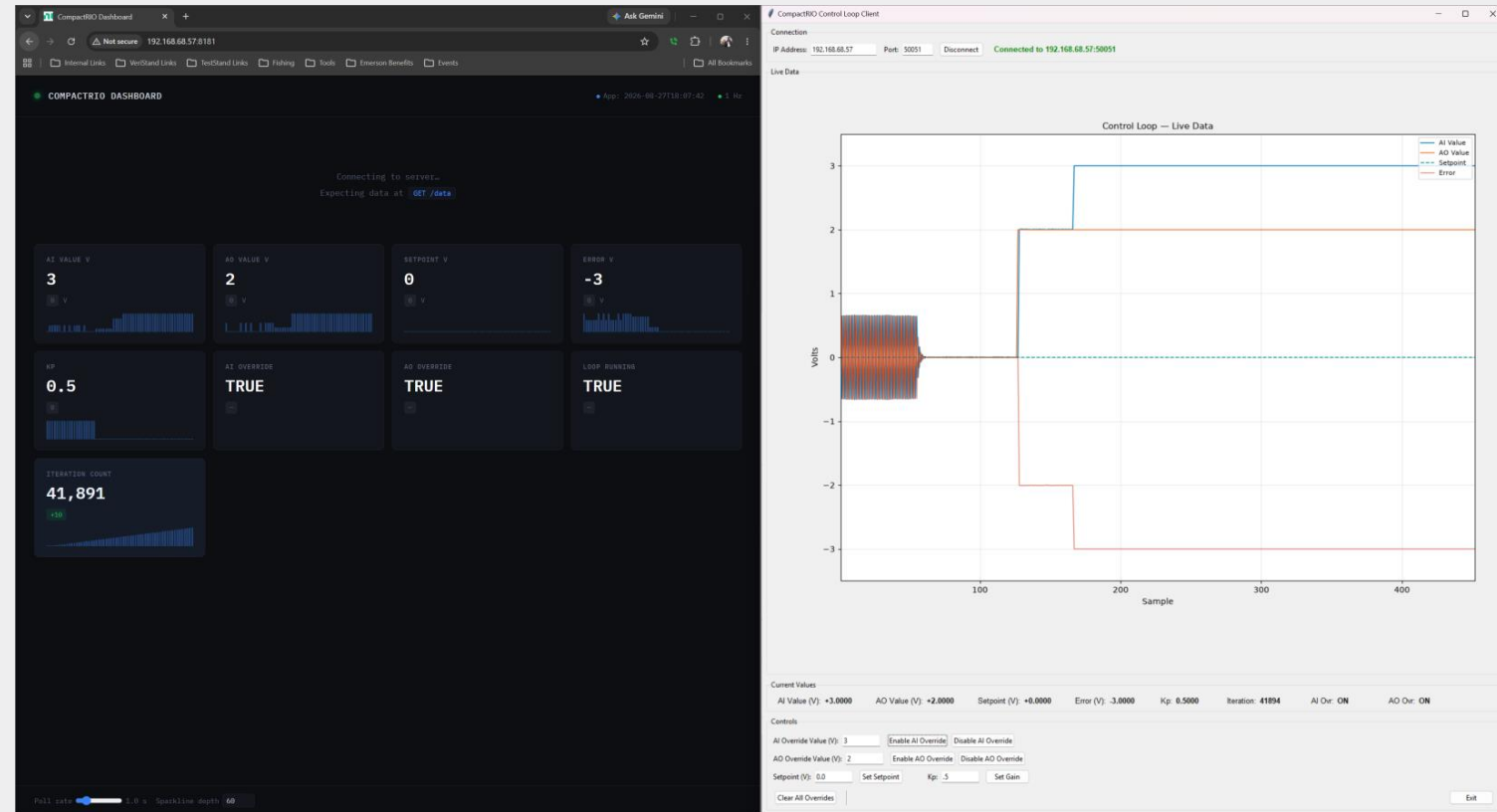
Vibe coded:

- cRIO Control App
- Locally hosted webpage
- gRPC Interface
- Windows gRPC GUI

Already:

- Installed drivers / python / venv
- Moved application to cRIO
- Validated app function

Now need set as executable / run on reboot...



Demo: Full Application

Review Code

control_loop.proto → Communication Definition

*_pb2.py & *_pb2_grpc.py → Communication API generated from Proto

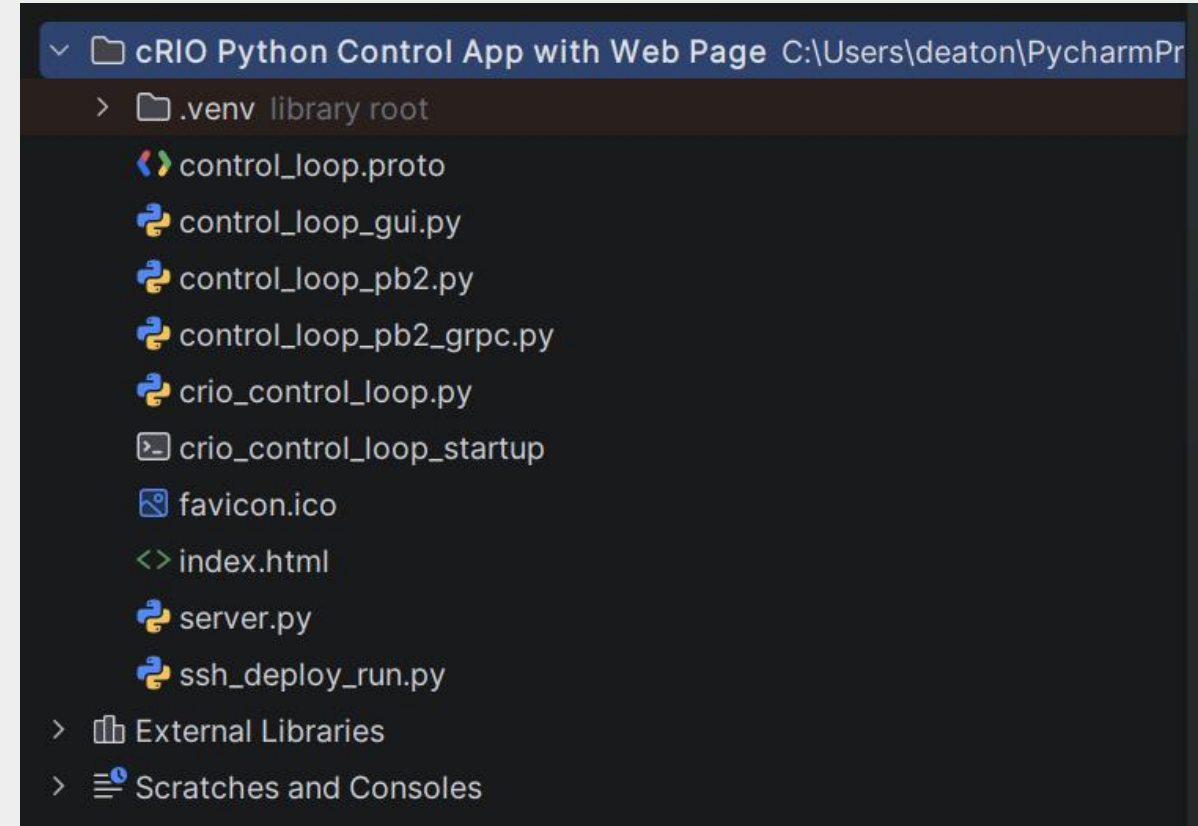
crio_control_loop.py → Main App

- Calls server.py
- Calls index.html
- Calls favicon.ico

crio_control_loop_startup → startup script for SysV

ssh_deploy_run.py → Host app moves crio files and remotely executes

control_loop_gui.py → Host gRPC GUI



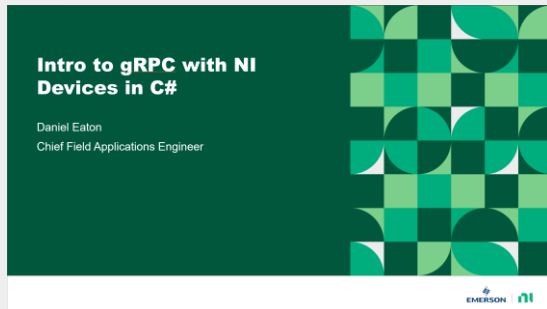
gRPC Lunch and Learn Available!!

Want more gRPC?!?

LabVIEW gRPC: <https://github.com/NISystemsEngineering/ni-LunchAndLearn-LabVIEWgRPCFundamentalsExamples>



NI Drivers gRPC: <https://github.com/NISystemsEngineering/ni-LunchAndLearn-gRPCDeviceOverviewExamples>



Demo: Full Application

Pro Tip: Add simple wait to start of app...

cRIO drivers need time to initialize...

Write code that checks if drivers are ready or...wait

```
def main() -> None: 1 usage
    time.sleep(30)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s %(levelname)-8s %(message)s",
    )
```

Demo: Full Application

Running from Windows

(assumes venv is setup)

From Windows execute:

Run command (in Setup Info.txt)

- Files are moved to cRIO
- Application started remotely
- Wait (30 seconds)

Connect to the webpage in Chrome

- IPAddress:8181

Start the Python gRPC Client:

- control_loop_gui.py

In terminal: Ctrl C stops

```
(.venv) PS C:\Users\deaton\PycharmProjects\cRIO Python Control App with Web Page> python ssh_deploy_run.py --host 192.168.68.57 --user admin --files control_loop_
crio_control_loop.py
Password for admin@192.168.68.57:
Connected to admin@192.168.68.57:22

Transferring files _
control_loop_pb2.py -> /home/lvuser/criocontrolapp/control_loop_pb2.py (3,595 bytes) _ OK
control_loop_pb2_grpc.py -> /home/lvuser/criocontrolapp/control_loop_pb2_grpc.py (14,571 bytes) _ OK
crio_control_loop.py -> /home/lvuser/criocontrolapp/crio_control_loop.py (19,335 bytes) _ OK
server.py -> /home/lvuser/criocontrolapp/server.py (4,317 bytes) _ OK
index.html -> /home/lvuser/criocontrolapp/index.html (14,661 bytes) _ OK
favicon.ico -> /home/lvuser/criocontrolapp/favicon.ico (4,286 bytes) _ OK

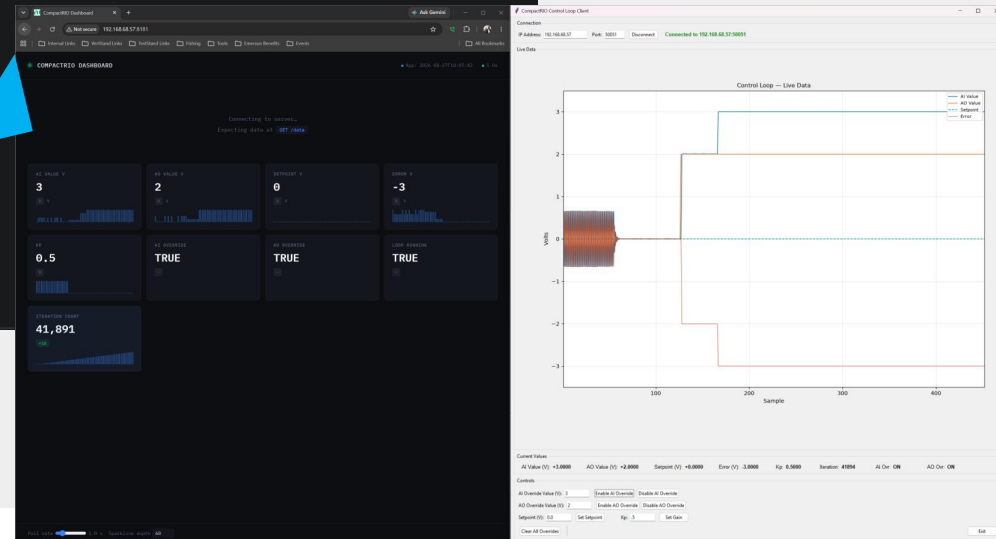
All files uploaded to /home/lvuser/criocontrolapp

Executing: cd /home/lvuser/criocontrolapp && source /home/lvuser/criocontrolapp/venv/bin/activate && python3 -u /home/lvuser/criocontrolapp/crio_control_loop.py
Press Ctrl-C to stop.

2026-08-27 19:23:56,476 INFO Launching dashboard server: /home/lvuser/criocontrolapp/venv/bin/python3 /home/lvuser/criocontrolapp/server.py --port 8181
2026-08-27 19:23:56,494 INFO gRPC server listening on [::]:50051
Dashboard running at http://192.168.68.57:8181
Reading variables from /tmp/crio_variables.json
2026-08-27 19:23:56,755 INFO AI task created on Mod5/ai0 (differential, on-demand)
2026-08-27 19:23:56,764 INFO AO task created on Mod3/ao0 (on-demand)
2026-08-27 19:23:57,009 INFO Control loop running at 10.0 Hz

>>> Ctrl-C caught - sending interrupt to remote process ...
^C
Shutting down.
2026-08-27 19:24:03,972 INFO Caught signal 2 - shutting down...
2026-08-27 19:24:03,996 INFO Control loop stopped; tasks closed.
2026-08-27 19:24:04,002 INFO Stopping dashboard server (pid 18517)...
2026-08-27 19:24:04,006 INFO Dashboard server stopped.
2026-08-27 19:24:04,006 INFO Server stopped.

Remote process exited with code 0
SSH connection closed.
```



Demo: Full Application

Create Startup Script

cRIO uses SysV

Create a bash file (no file extension needed)

- Shebang (#!/bin/bash) at the top
- Set python venv path
- Use Start-Stop-Daemon to launch
- Use PID (and pid file) to stop
- Use Log file to record standard outputs
- Use Linux EOL (if on Windows)

Example:

..\cRIO Control App with Web Page\Python Source\crio_control_loop_startup

```
1  #!/bin/bash
2
3  NAME="crio_control_loop_startup"
4  APP_DIR="/home/lvuser/criocontrolapp"
5  PYTHON_VENV="/home/lvuser/criocontrolapp/venv/bin/python"
6  SCRIPT="/home/lvuser/criocontrolapp/crio_control_loop.py"
7  LOG="/var/log/crio_control_loop_startup.log"
8  PIDFILE="/var/run/crio_control_loop_startup.pid"
9
10 do_start() {
11     echo "Starting $NAME"
12     export PYTHONUNBUFFERED=1
13     sleep 2
14     cd "$APP_DIR" || exit 1
15     start-stop-daemon --start --background --no-close --pidfile "$PIDFILE" --make-pidfile --chdir "$APP_DIR" --exec "$PYTHON_VENV" -- "$SCRIPT" $DAEMON_ARGS >> "$LOG" 2>&1
16 }
17
18 do_stop() {
19     echo "Stopping $NAME"
20     start-stop-daemon --stop --pidfile "$PIDFILE" --retry 10 --verbose
21     rm -f "$PIDFILE"
22 }
23
24 do_restart() {
25     do_stop
26     sleep 1
27     do_start
28 }
29
30 case "$1" in
31     start)
32         do_start
33         ;;
34     stop)
35         do_stop
36         ;;
37     restart)
38         do_restart
39         ;;
40     *)
41         echo "Usage: $0 {start|stop|restart}"
42         exit 1
43         ;;
44 esac
45
46 exit 0
```


Demo: Full Application

Register Startup Script

Move file to cRIO: /etc/init.d

Set cRIO file permissions:

- `chmod +x /etc/init.d/crio_control_loop_startup`

Test bash file

- `/etc/init.d/crio_control_loop_startup start`
- `/etc/init.d/crio_control_loop_startup stop`

Make bash file part of start up sequence:

- `/usr/sbin/update-rc.d -f crio_control_loop_startup defaults`

Reboot

NOTE: Stopping startup script: `/usr/sbin/update-rc.d -f crio_control_loop_startup remove`

```
admin@NI-cRIO-9047:~# /etc/init.d/crio_control_loop_startup start
Starting crio_control_loop_startup
admin@NI-cRIO-9047:~# /etc/init.d/crio_control_loop_startup stop
Stopping crio_control_loop_startup
Stopped process in pidfile '/var/run/crio_control_loop_startup.pid' (pid 26305).
admin@NI-cRIO-9047:~# /usr/sbin/update-rc.d -f crio_control_loop_startup defaults
Adding system startup for /etc/init.d/crio_control_loop_startup.
admin@NI-cRIO-9047:~# /usr/sbin/update-rc.d -f crio_control_loop_startup remove
update-rc.d: /etc/init.d/crio_control_loop_startup exists during rc.d purge (continuing)
Removing any system startup links for crio_control_loop_startup ...
/etc/rc0.d/K20crio_control_loop_startup
/etc/rc1.d/K20crio_control_loop_startup
/etc/rc2.d/S20crio_control_loop_startup
/etc/rc3.d/S20crio_control_loop_startup
/etc/rc4.d/S20crio_control_loop_startup
/etc/rc5.d/S20crio_control_loop_startup
/etc/rc6.d/K20crio_control_loop_startup
admin@NI-cRIO-9047:~#
```

Demo: Full Application

Power cycle the cRIO

Wait roughly 30 seconds

Connect to the webpage in Chrome

- IPAddress:8181

Start the Python gRPC Client:

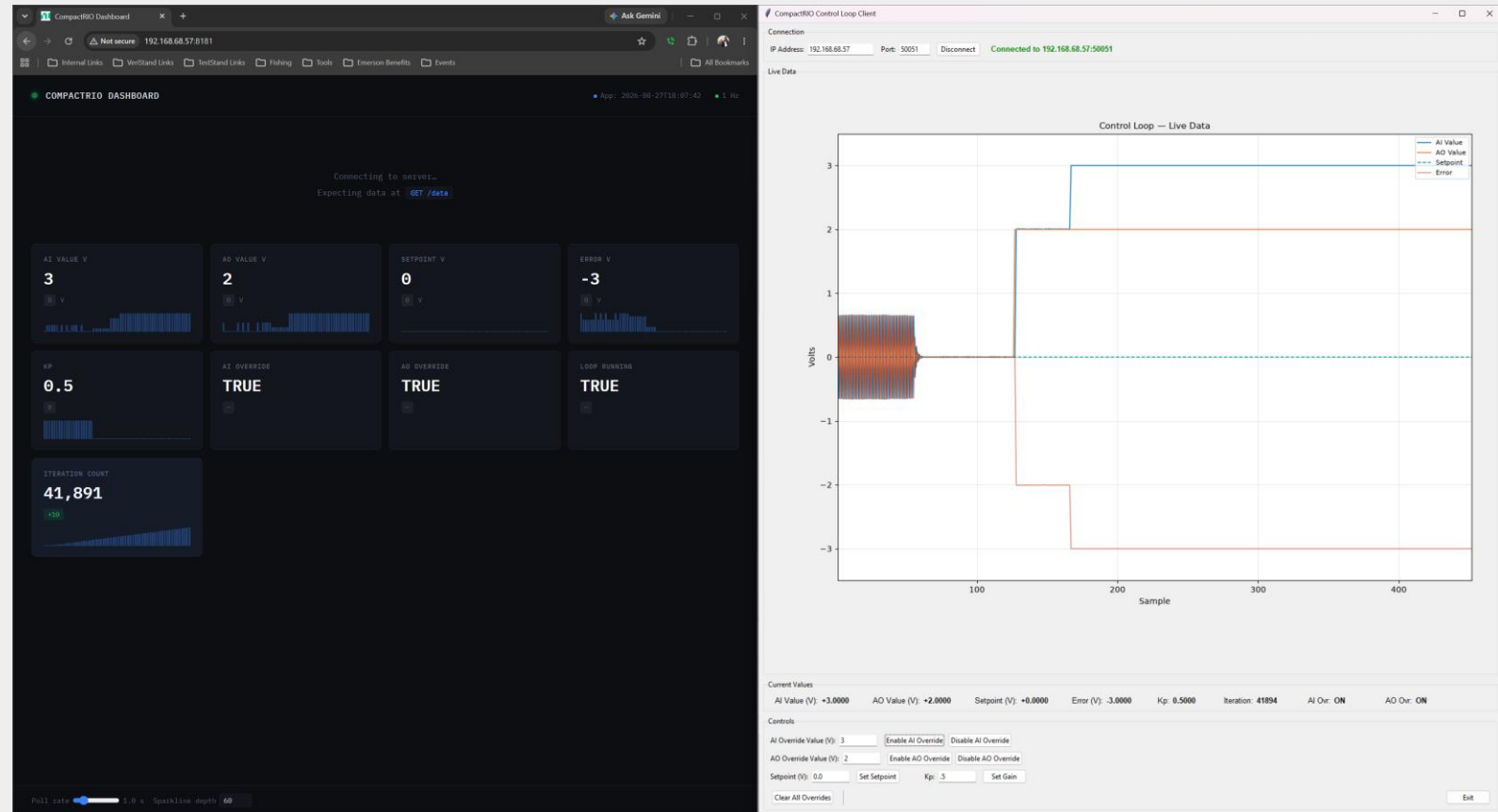
- control_loop_gui.py

Confirm pid file on cRIO:

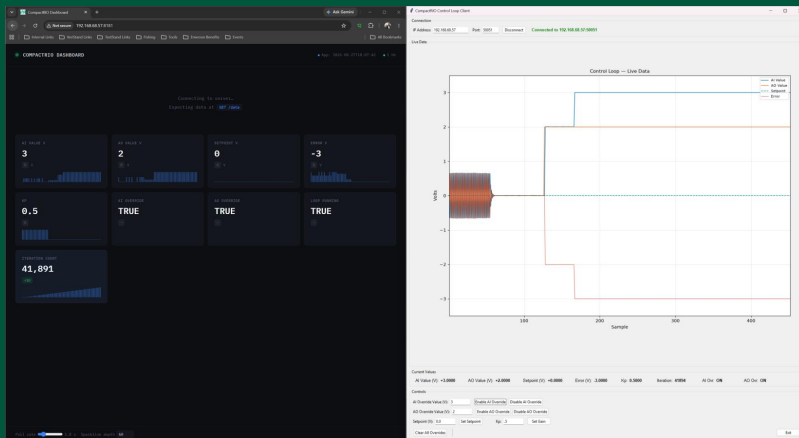
- /var/run/crio_control_loop_startup.pid

Confirm log file on cRIO:

- /var/log/crio_control_loop_startup.log



DEMO: Full Application



cRIO uses SysV

Wait for drivers to initialize

gRPC is amazing...

Vibe coding is amazing...

Offline Management

NI Drivers

NI Linux Real Time Offline Installers:

https://www.ni.com/en/support/downloads/software-products/download.ni-linux-real-time-offline-installation-support.html?srsId=AfmBOoo5KuqCqTZ94DYXrkMwt5_9bXWtGFNQMk2m6hJa2JWqbM3Xa6TF#607120



NI Linux Real-Time Offline Installation Support

The NI Linux Real-Time Offline Installation Support provides offline access to drivers for NI Linux Real-Time targets.
[+ Read More](#)

DOWNLOADS

Supported OS	Windows	View Release Notes
Version	2026 Q3 	
Included Editions	Full	
Application Bitness	32-bit and 64-bit	

NI Linux Real-Time Offline Installation Support
2026 Q3

Release Date
Jul/23/2026

Included Versions
2026 Q3

> **Supported OS**

> **Checksum**

[Log in to download](#)

File Size
8.59 MB

Offline Management

Linux Packages

NI Uses OPKG & IPKs

Get a list of all installed packages:

- `opkg list-installed | awk '{ print $1 }' > installed_packages.txt`

Download ipk for each package:

- `xargs -a installed_packages.txt opkg download`

Compress the packages

- `tar -czf offline_ipks.tar.gz /path/to/ipks`

Offline Management

Linux Packages

Move compressed packages to new target

Extract

- `tar -xf offline_ipks.tar.gz`

Install

`for file in *.ipk; do opkg install "$file"; done`

Offline Management

Linux Packages – On Connected Target

```
mkdir /srv/opkg-repo
```

```
cd /srv/opkg-repo
```

```
opkg list-installed | awk '{ print $1 }' > installed_packages.txt
```

```
xargs -a installed_packages.txt opkg download
```

```
cd ..
```

```
mkdir opkg-repo-compressed
```

```
cd opkg-repo-compressed
```

```
tar -czf offline_ipks.tar.gz /srv/opkg-repo
```

```
cp /srv/opkg-repo/installed_packages.txt /srv/opkg-repo-compressed
```

Offline Management

Linux Packages – On Disconnected Target

(move compressed file to /)

```
tar -xf offline_ipks.tar.gz
```

```
cd /srv/opkg-repo
```

```
for file in *.ipk; do opkg install "$file"; done
```


Offline Management

Collect Python Packages

Download packages with dependencies

In venv, get list of packages

- `pip freeze > requirements.txt`

Download packages (with online computer)

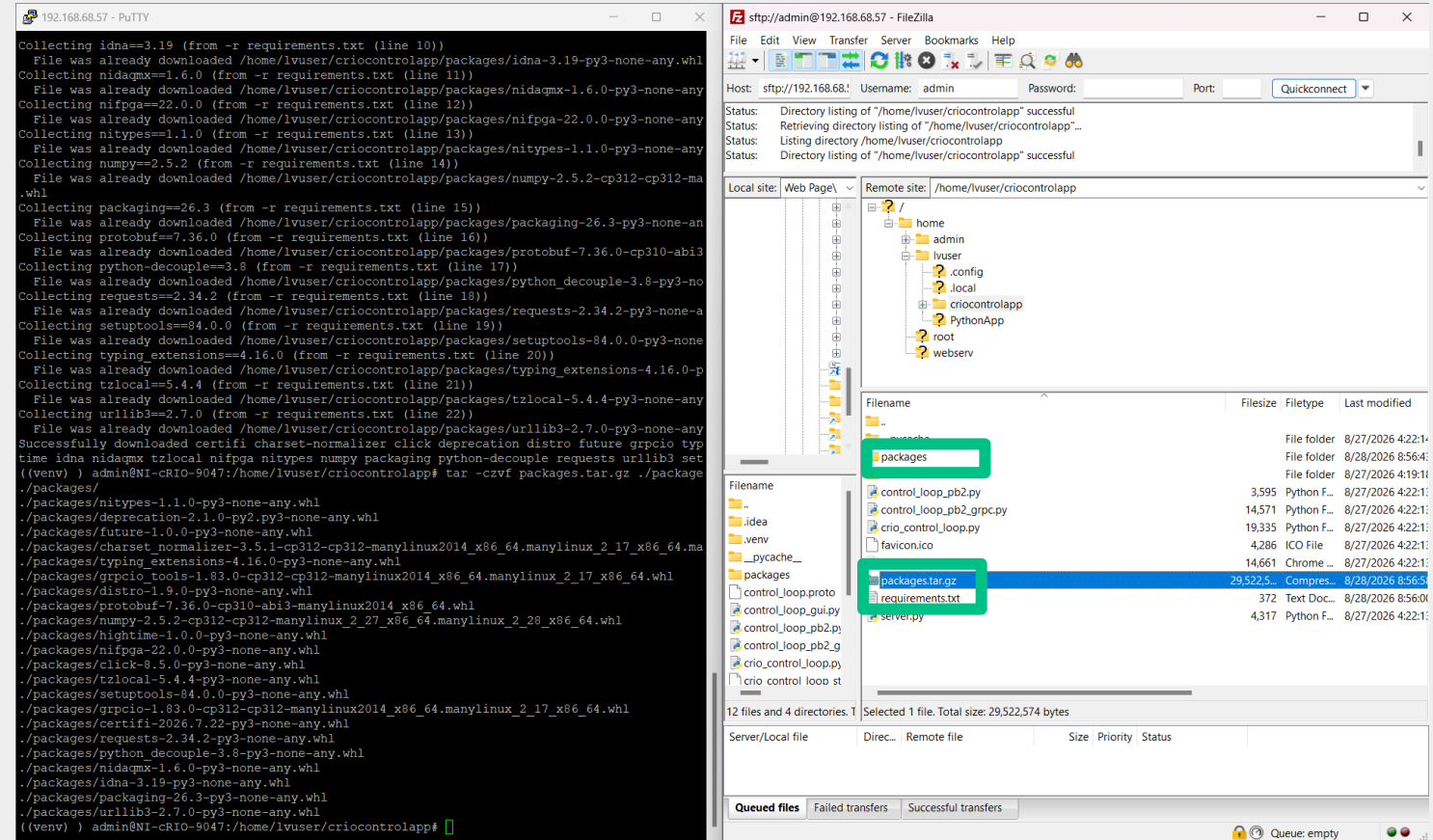
- `pip download -d ./packages -r requirements.txt`

Compress all packages

- `tar -czvf packages.tar.gz ./packages`

If you want an individual package

- `pip download grpcio-tools -d ./packages`



Offline Management

Install Python Packages

Move requirements.txt and packages.tar.gz to offline machine

Create venv (python is already on offline machine)

- `python -m venv venv`
- `source venv/bin/activate`

Uncompress packages

- `tar -xzf packages.tar.gz`

Install packages

- `pip install --no-index --find-links=./packages -r requirements.txt`

Move your app over and run

Filename	Filesize	Filetype	Last modified
..			
packages.tar.gz	29,522,5...	Compres...	8/28/2026 8:58:0...
requirements.txt	372	Text Doc...	8/28/2026 8:57:4...

Filename	Filesize	Filetype	Last modified
..			
venv		File folder	8/28/2026 9:03:1...
packages.tar.gz	29,522,5...	Compres...	8/28/2026 8:58:0...
requirements.txt	372	Text Doc...	8/28/2026 8:57:4...

Filename	Filesize	Filetype	Last modified
..			
packages		File folder	8/28/2026 8:56:4...
venv		File folder	8/28/2026 9:03:1...
packages.tar.gz	29,522,5...	Compres...	8/28/2026 8:58:0...
requirements.txt	372	Text Doc...	8/28/2026 8:57:4...

And Now...

Go and Vibe!!!

Questions?



Daniel Eaton

Chief Field Applications Engineer

CLA, CLED, 20 years “doing” w/ NI

Daniel.eaton@emerson.com



We are here to help!

- Pull us in to design conversations
- Let us help de-risk projects
- Tell us what you want to learn about next